PRODUCT REQUIREMENTS · PHASE 16

Invoices & Clinical Documents

Five gaps, one phase. The cashier can bill but cannot hand over a receipt — and when one exists, it can only be collected at a counter. The doctor can write an encounter but cannot see the lab result in the patient's hand. The clinic tracks licence numbers with no scan behind them and no warning before they lapse. And whatever the clinic publishes, one person writes it and the same person ships it.

STATUS	DATE	PHASE	EFFORT	DEPENDS ON
Draft — for review	30 August 2026	P16-T01 … P16-T41	≈248 pts · 12–14 sprints	P8 EMR · P9 Billing · P15 Docs · SJ-21 · GOWA · SMTP

01

Problem & context

The invoice ends at the screen

`Invoice`, `InvoiceItem`, `Payment` and per-day `InvoiceCounter` numbering all ship today, and the billing workspace renders a bill on screen. There is no document. Grepping the API for PDF generation returns only `pdf-parse`, which *reads* uploaded PDFs for the retrieval corpus — nothing writes one.

The workaround is a screenshot or a hand-written *kwitansi*. That costs the clinic three ways: the patient cannot claim reimbursement from a private insurer without a document showing the clinic's identity and a line-item breakdown; the clinic's brand is absent from the one artefact the patient keeps; and a hand-written receipt does not reconcile against `Payment` rows, so the cashier report and the paper trail disagree.

Clinics will not accept one hard-coded layout. Logo, address, practice licence number, footer disclaimer, signature block and the *terbilang* (amount-in-words) line all differ per clinic, and a support ticket per layout change is not a product.

The doctor is blind to the patient's own paperwork

The encounter workspace shows vitals, diagnoses, procedures and prescriptions — all structured data typed into HMS. It shows nothing that arrived as a file: the outside-lab result, the radiology report from the hospital next door, the referral letter, the signed consent form. Those exist today as paper the patient carries, or as a photo on a nurse's phone.

> **ALREADY ANTICIPATED IN THE SCHEMA**
>
> The `Document` docstring says the table serves "the clinic FAQ corpus, the Phase 15 retrieval corpora, personal knowledge bases, and **the future patient/doctor document features**", and `DocumentOwnerType` already declares `PATIENT` and `DOCTOR` values that nothing writes yet. This PRD is the feature that docstring was written for.

A doctor has nowhere private to keep their own paperwork, and the clinic still cannot see an expiry coming

Two separate problems have been getting confused with each other.

The doctor's problem. Under Indonesian practice a doctor's professional file is not one document but a stack: STR from the Konsil, one SIP per practice location, ijazah, Sertifikat Kompetensi from the kolegium, IDI branch recommendation, KTP, *surat keterangan sehat*, plus the CV, CME/P2KB certificates and malpractice cover that credentialing checklists call for. It lives on their phone and in their email. HMS gives a doctor a private knowledge base for the papers they *read* and nowhere at all for the papers that are *about them*.

The clinic's problem. `DoctorLicense` stores `type`, `licenseNumber`, `issuedAt` and `expiresAt`, indexed on `expiresAt` — the schema comment says clinics "must track SIP expiry for licensing audits". Nothing warns anyone that an SIP lapses in three weeks.

These look like one feature and are not. The doctor's problem is storage that nobody else can open. The clinic's problem is **a number and a date** it already holds. Solving the second by reading the first is what an earlier draft of this PRD did, and it was wrong: it made a doctor's KTP an admin screen.

The receipt still has to be handed over

The clinic already runs a WhatsApp number through a self-hosted GOWA bridge and an SMTP account through the shared mail module. Patients book appointments over that WhatsApp number. But the one document they most want to keep — the bill — is the one thing that can only be obtained by standing at a counter.

The two transports are in production; what is missing is narrow and specific. `WhatsappGatewayService` has exactly one method, `sendText`. `SendMailRequest` has no attachment field. Nothing records that a document was sent to anyone. And

`PatientProfile.email` is free text a clerk typed once, which nothing has ever verified — so “just email it” is a misdelivery waiting to happen unless the delivery shape accounts for that.

Publishing is a one-click act with no second pair of eyes

The clinic publishes two kinds of thing under its own name, and both go live on one person's click.

The invoice template is the clinic's outgoing document — its letterhead, its licence number, its charges. Under E1 as specified, one admin edits it and one admin publishes it, and the next patient's receipt carries whatever they typed.

The clinic FAQ corpus is worse. `/admin/clinic-corpus` ships today: an admin uploads a document, sets `visibility`, confirms, and the ingestion worker embeds it — after which **the chatbot can retrieve and cite it to patients**. The schema is explicit about the stake: `DocumentVisibility` exists because “a staff-only SOP must never surface in a patient answer, and this column is the enforcement.” One person sets that column. Nobody else reads it before the chatbot starts quoting the result.

For a one-doctor klinik that is correct, and anything else would be friction. For a clinic with a medical director, a finance lead and three admins, it is a gap: the person who writes the layout should not be the only person who ever sees it before a patient does. Neither answer is right for both clinics, which is why this is a switch rather than a decision the product makes on their behalf.

The repo already knows the shape. `appointment.approve:any` exists as a permission distinct from writing, and `APPOINTMENT_APPROVED` / `APPOINTMENT_REJECTED` are already notification types — special appointment requests are approval-gated today. E5 applies that pattern to publishing.

02

Goals & success metrics

Measured 90 days after launch.

G-1	G-2	G-3
Every settled bill produces a document the patient keeps	Clinics own their invoice layout without engineering	Invoice documents are trustworthy
≥95%	0 h	<1%
of PAID invoices with a PDF · from 0%	eng. time per layout change · from ~4 h	reissued or corrected PDFs

G-4

Doctors see outside paperwork in context

≥40%

of encounters open ≥1 document · from 0%

G-5

Uploading beats photocopying

<45 s

p50 click to confirmed document

G-6

No licence lapses unnoticed

0

licences lapsing without an admin warning · from 100%

G-7

A doctor's file is seen only by people they chose

0

opened by a non-owner without a live, owner-created share

G-8

Receipts reach patients without a counter handover

≥60%

of PAID invoices delivered · from 0%

G-9

Delivery is reliable enough to depend on

≥98%

reaching SENT within 5 minutes

G-10

Nothing is sent to someone who did not agree

0

sends without consent or a verified number

G-11

Nothing is published without the review the clinic asked for

100%

clinic documents going live under an active policy carry an approval record

G-12

The gate does not become a bottleneck

<1 day

median submission to decision

03

Non-goals

Half of a PRD's value is the list of things the team is allowed to skip.

NOT DOING	WHY	MIGHT RETURN AS
e-Meterai (electronic stamp duty)	Needs a Peruri-licensed distributor account and per-stamp billing; the layout only needs to *reserve* the stamp area	Post-phase spike (OQ-3)
Partial payments / instalments	`Payment.invoiceId` is `@unique` by deliberate v1 scope; changing it is a billing change, not a document change	Billing v2
~~Sending invoices by email or WhatsApp~~ — **now in scope as E4**	Moved in at the product owner's direction. The consent and misdelivery questions it raises are answered in E4 rather than deferred	—
DICOM viewing or a PACS bridge	Storing a radiology *report* is cheap; rendering imaging studies is a product of its own	Not scheduled
OCR of uploaded lab values into structured EMR	Real value, but an ML feature with a clinical-safety bar. Storing and showing the file is the 80%	Phase 15 retrieval track
Patient self-upload from the portal	Adds a near-unauthenticated write path into the clinical record; staff-mediated upload first	E2 v2
Inline in-app preview of stored files	`docs/security/file-uploads.md` forbids rendering stored files in the app or API origin; needs a dedicated user-content origin	E2 v2, blocked on SJ-1
General clinic DMS (contracts, SOPs, non-doctor HR)	Scope discipline. The clinic corpus already handles SOPs for the chatbot	Evaluate after E3
Multi-clinic template inheritance	One deployment per clinic today; multi-tenancy is still a proposal	Whenever tenancy lands
Clinic review, verification or approval of a doctor's own documents	The owner decides who sees their document; there is no reviewer whose sign-off it needs. The clinic's compliance need is met from `DoctorLicense` instead	Not planned
Recipients re-sharing, or any share-my-whole-vault switch	A share is a key to one door. Transitive sharing means the owner no longer knows the audience, which is the one property that makes the feature safe to use	Not planned
Admin-initiated access requests (“ask this doctor for their STR”)	An in-product request is pressure with an audit trail. The ask happens between people; the product only records the grant	Reconsider only if clinics report the gap
~~Sending clinical documents over WhatsApp or email~~ — **now in scope**	Moved in on RQ-2: the patient often receives the physical result first, so withholding the digital copy protects nobody. Dual delivery sends to patient and doctor at once, removing the failure the exclusion guarded against	—
Payment links / collecting money in the message	A payment-gateway integration with its own PCI and reconciliation surface, not a document-delivery feature	Billing v2
Bulk or marketing messaging on the clinic number	The fastest route to a banned WhatsApp number, and not what this channel is for	Never on this channel
SMS as a third delivery channel	No SMS provider is integrated, and Indonesian A2P SMS is costlier and less reliable than either channel already available	If a clinic asks and a provider is chosen
Multi-step or sequential approval chains (level 1, then level 2)	One round with N approvers covers every clinic size we have seen; chains add a state machine nobody has asked for	If a clinic with a real two-tier governance model asks

NOT DOING	WHY	MIGHT RETURN AS
Approval gates on patient (E2) or doctor (E3) documents	Those are records about a person, not clinic publications. Clinical release is a clinician's judgement about one patient; credential verification is checking someone else's evidence. Neither belongs to a governance approver	Not planned; see E5
Approval on invoice *issuing* or payment	E5 gates publishing a document template, not the billing acts themselves. Gating money movement is a different feature with different reviewers	Billing v2, if asked

04

Personas & primary journeys

PERSONA	JOURNEY THIS PRD SERVES
Cashier `ADMIN today`	Takes payment → records it → prints or downloads the PDF receipt → hands it to the patient. Reprints it a week later on request.
Clinic owner	Opens the template editor once at onboarding, drops in logo, address and licence number, inserts variables, previews with sample data, publishes. Returns twice a year.
Doctor	Opens an encounter, sees a Documents panel newest-first, downloads the lab PDF, reads it, writes the note. Separately keeps their own STR, certificates and CV in a private vault nobody else can open.
Front desk / nurse	Scans or photographs the paperwork the patient brought, uploads it against the patient, tags a category, optionally links it to today's encounter.
Clinic admin / HR	Works the licence-expiry list every month from `DoctorLicense` numbers and dates. Sees a doctor's actual STR scan only if that doctor shared it, and only that one document.
Approver `medical director / owner`	Holds `document-approval.decide:any` without necessarily authoring anything. Works one queue at `/admin/approvals` covering every clinic-owned document: opens a submission, sees the frozen payload and what changed, approves or sends it back with a reason.
Patient	Receives the receipt on WhatsApp, where they already talk to the clinic, and keeps it on their phone for reimbursement. Can reply `BERHENTI` to make it stop.
Records officer	Answers a surveyor's question about who looked at a patient's file, and a payer's question about what a bill contained. Needs the audit trail and the immutable snapshot, not the live data.

05

Current state in the codebase

EXISTS AND IS REUSED AS-IS

- `Invoice`, `InvoiceItem`, `Payment`, `ServiceTariff`, `InvoiceCounter`, and the issue / payment / void routes in `billing/controller/invoice.controller.ts`
- `Document`, `DocumentChunk`, and the unused `PATIENT` / `DOCTOR` values on `DocumentOwnerType`

- Presigned upload and download, server-minted UUID keys, magic-byte validation at confirm — `common/storage` and `uploaded-document-guard.service.ts`
- CASL `PermissionsGuard` with `resource.action:scope`, seeded in `prisma/seed.sql`
- `GowaWhatsappAdapter` and `WahaWhatsappAdapter` behind the provider-neutral `WhatsappGatewayService` port, selected by `WA_GATEWAY_KIND`, with a cross-adapter conformance suite and send pacing in `WhatsappBridgeHttpClient`
- `MailService` over nodemailer, provider-neutral through six `MAIL_*` variables, with a `log` transport for local dev
- `ChannelPatientLink` + `ChannelOtpChallenge` — a patient phone number proven by OTP or contact-share, the only trustworthy delivery destination the system has
- **An approval pattern to copy.** `appointment.approve:any` is already a permission distinct from writing, `approveAppointment` / `rejectAppointment` already take a mandatory rejection reason, and `APPOINTMENT_APPROVED` / `APPOINTMENT_REJECTED` are already notification types. E5 is that pattern applied to publishing, not a new invention.
- **The clinic corpus surface E5 gates.** `/admin/clinic-corpus` with `clinic-documents/` components, `DocumentAdminController` (upload-url, confirm, ingest, patch, delete), and a `DocumentService` that pins `ownerType = CLINIC` and sets `ingestStatus = PENDING` on confirm. E5 moves that one assignment behind a decision.
- `DoctorLicense` (STR / SIP, `expiresAt` indexed), `DoctorEducation`, `DoctorPatient` assignment, the `Notification` bell feed, and `AuditLog` with `READ` as a first-class action

MISSING

- Any PDF *writer*. `pdf-parse` reads; nothing produces.
- Any rich-text editor in `apps/web` or `packages/ui`.
- Any clinic-profile record. Clinic identity exists only as `CS_CLINIC_NAME`, an env var used by the customer-service prompt.
- Any link from a `Document` to a `PatientProfile`, `Encounter`, `Admission` or `DoctorLicense`.
- Any image MIME support. The bucket allowlist is exactly PDF, Markdown and plain text, narrowed on purpose under SJ-21.
- Any way to send a **file**. `WhatsappGatewayService` exposes only `sendText`; `SendMailRequest` is `{ to, subject, text, html }` with no attachment field.
- Any delivery record, delivery consent, or verification of `PatientProfile.email`.

MUST NOT BREAK

- **Financial snapshots.** `InvoiceItem` prices and `Invoice.totalAmount` are stored, not derived, so a tariff change never rewrites an issued bill. The rendered invoice inherits this rule — see `FR-E1-09`.
- **Retrieval scoping.** `document.read:own` for a doctor means their personal corpus and nothing else. E2 and E3 must not widen it — see `FR-E3-11`.

- The 5 MiB / three-MIME upload posture is a security decision, not a default. Widening it is part of this PRD's cost, stated in `NFR-SEC-02`.
- **The narrowness of `WhatsappGatewayService`.** Its docstring records that one method after two implementations is the evidence the port shape was right, and D-CS-01 names the official Cloud API as the endgame. E4 adds exactly one member, and only because all three implementations can satisfy it.
- **Conversation replies keep priority on the WhatsApp queue.** Invoice sends share `WA_GATEWAY_SEND_PACING_MS` and must never delay a booking confirmation — `NFR-PERF-04`.

EPIC 1 OF 5 · 57 POINTS

Invoice PDF with a clinic-customizable template

> The cashier hands the patient a PDF that looks like the clinic's own document, produced from a layout the clinic edited themselves, and that never changes after it is issued.

Functional requirements

ID	PRI	REQUIREMENT
FR-E1-01	MUST	An admin can create, edit, duplicate and archive named invoice templates. Exactly one template is the clinic default at any time.
FR-E1-02	MUST	The template body is edited in a WYSIWYG rich-text editor supporting headings, bold / italic / underline, ordered and unordered lists, alignment, tables, horizontal rules, page breaks and a logo image.
FR-E1-03	MUST	The editor offers a searchable variable palette. Inserting a variable places a non-editable inline chip showing its label; the stored document holds a machine token, not the label text.
FR-E1-04	MUST	A **repeating block** variable renders invoice line items as rows. The author controls which columns appear and their order.
FR-E1-05	MUST	Template settings cover paper size (A4, A5, Letter), orientation, margins, and whether a header / footer repeats on every page. Thermal roll is out — clinics print on lightweight sheet stock, so sheet sizes are the whole requirement.
FR-E1-06	MUST	**Preview with sample data** renders the current draft against a fixture invoice, in-editor, without touching real patient data.
FR-E1-07	MUST	A template has DRAFT and PUBLISHED states. Only a published version is used for a real invoice. Publishing creates an immutable version row.
FR-E1-08	MUST	Variables resolve against real invoice data at render time. An unresolvable variable renders empty, never as the raw token, and is counted in a render warning.
FR-E1-09	MUST	**Issuing an invoice snapshots the render.** The template version id and the fully resolved variable values are persisted with the invoice. Re-rendering later reproduces the identical document even if the template has since changed.
FR-E1-10	MUST	The cashier can download and print the PDF from the invoice detail view, for any invoice in ISSUED or PAID state.
FR-E1-11	MUST	A VOID invoice renders with a visible VOID / BATAL watermark and the void reason; it is never silently reissued under the same number.
FR-E1-12	MUST	The total is also rendered in Indonesian words (*terbilang*): Rp 275.000 → “dua ratus tujuh puluh lima ribu rupiah”.
FR-E1-13	SHOULD	The layout reserves a *materai* area that appears only when the total exceeds a configurable threshold (default Rp 5.000.000). It is a **placement for a physical stamp** — e-Meterai stays out of scope, confirmed by the product owner.
FR-E1-14	SHOULD	The footer can carry a QR code encoding a verification URL for the invoice number, so a payer can check a receipt is real.
FR-E1-15	SHOULD	A clinic profile record supplies the `clinic.*` variables, replacing `CS_CLINIC_NAME` as the single source of clinic identity.
FR-E1-16	COULD	Templates export and import as a JSON bundle, so a layout can move between environments or seed a new clinic.

User stories

US-E1-01 Cashier prints a receipt

5 pts

> As a **cashier**, I want to download the invoice as a PDF after recording payment, so that the patient leaves with a receipt they can claim on.

GIVEN an invoice in PAID state WHEN the cashier clicks *Download PDF* THEN a PDF returns within 5 s, named `INV-<invoiceNumber>.pdf`, containing the clinic header, patient name and MRN, every line item with quantity / unit price / amount, the total in figures and in words, the payment method and reference, and the cashier's name.

GIVEN the same invoice a week later WHEN the cashier downloads it again THEN the file is byte-identical to the first download.

GIVEN an invoice still in DRAFT THEN *Download PDF* is disabled with the hint “Issue the invoice first”.

US-E1-02 Owner brands the invoice

8 pts

> As a **clinic owner**, I want to edit the invoice layout myself and drop in my logo and address, so that the receipt looks like my clinic's document.

GIVEN the template editor WHEN the owner uploads a PNG logo under 1 MB and publishes THEN the next issued invoice renders that logo in the header at the declared size, embedded in the PDF rather than linked.

GIVEN an unsaved draft WHEN the owner navigates away THEN they are warned that unpublished changes will be lost.

GIVEN a published template WHEN the owner edits and republishes THEN invoices issued earlier still render from their snapshotted version, and only later ones use the new layout.

US-E1-03 Owner inserts system variables

8 pts

> As a **clinic owner**, I want to insert placeholders like patient name and total, so that each invoice fills itself in.

GIVEN the editor **WHEN** the owner opens the palette and types “mrn”
THEN `{{patient.mrn}}` — *Nomor Rekam Medis* — appears with its description and a sample value.

GIVEN an inserted variable chip **WHEN** the owner places the caret inside it and types
THEN the chip is not split; it is atomic and can only be deleted whole.

GIVEN a template with `{{items}}` configured as description / qty / unit price / amount
WHEN it renders against a 7-line invoice **THEN** the table has 7 body rows in item order, a header row on every page, and one totals row.

US-E1-04 Owner previews before publishing

3 pts

> As a **clinic owner**, I want to see the layout with realistic data before I publish, so that I do not discover a broken layout on a patient's receipt.

GIVEN a draft template **WHEN** the owner clicks *Preview* **THEN** a rendered preview appears using a fixture invoice containing a long patient name, 12 line items forcing a page break, a zero-price item, and a total above the materai threshold.

GIVEN a preview render **THEN** no request touches real patient data and the preview is never persisted to the invoice document store.

US-E1-05 Voided invoice is unmistakable

3 pts

> As a **records officer**, I want a voided invoice's PDF to say so on its face, so that a cancelled bill cannot be presented as a valid one.

GIVEN an invoice voided with reason “wrong patient” **WHEN** its PDF is downloaded **THEN** a diagonal BATAL / VOID watermark covers each page and the reason and voiding user are printed in the footer.

`US-E1-06` **Layout survives odd data** `5 pts`

> As an **engineer**, I want the renderer to behave predictably on hostile input, so that one long field does not produce an unusable document.

GIVEN a 120-character patient name **THEN** it wraps within its cell and never overlaps another field.

GIVEN an invoice with 60 line items **THEN** the table paginates, the header repeats, and the totals block is not orphaned on a page of its own.

GIVEN a variable resolving to null — `{{patient.nik}}` on a chat-created draft patient — **THEN** the field renders empty, one warning is reported, and the PDF is still produced.

Data model delta

Additive only. No backfill: an invoice issued before this phase has no snapshot, so its PDF binds the *current* default template on first render and is snapshotted then, with a `renderWarnings` entry recording the retroactive binding. That is stated on the download button, not hidden.

```
/// Clinic identity for anything the clinic puts its name on. One row per
/// deployment today; becomes tenant-scoped when multi-tenancy lands.
model ClinicProfile {
  id             String   @id @default(uuid()) @db.Uuid
  name           String
  legalName      String?  @map("legal_name")
  address        String?
  phoneNumber    String?  @map("phone_number")
  email          String?
  licenseNumber  String?  @map("license_number")
  taxId          String?  @map("tax_id")
  /// Object key in the private bucket. Never a URL (D-018).
  logoStorageKey String?  @map("logo_storage_key")
  createdAt      DateTime @default(now()) @map("created_at")
  updatedAt      DateTime @updatedAt @map("updated_at")

  @@map("clinic_profiles")
}

enum DocumentTemplateKind   { INVOICE }
enum DocumentTemplateStatus { DRAFT  PUBLISHED  ARCHIVED }
enum PaperSize              { A4  A5  LETTER }

/// The editable template. `contentHtml` is server-sanitised on every write;
/// it is never rendered in the app origin, only inside the isolated renderer.
model DocumentTemplate {
  id          String                @id @default(uuid()) @db.Uuid
  kind        DocumentTemplateKind
```

```
  name        String
  status      DocumentTemplateStatus @default(DRAFT)
  isDefault   Boolean                @default(false) @map("is_default")
  createdById String?                @map("created_by_id") @db.Uuid
  createdAt   DateTime               @default(now()) @map("created_at")
  updatedAt   DateTime               @updatedAt @map("updated_at")
  deletedAt   DateTime?              @map("deleted_at")

  versions DocumentTemplateVersion[]

  /// Partial unique index in the migration: at most one default per kind
  /// WHERE is_default AND deleted_at IS NULL.
  @@index([kind, status])
  @@map("document_templates")
}

/// An immutable published snapshot. A rendered invoice points here, never at
/// the mutable template — the InvoiceItem snapshot rule applied to layout.
model DocumentTemplateVersion {
  id            String   @id @default(uuid()) @db.Uuid
  templateId    String   @map("template_id") @db.Uuid
  versionNumber Int      @map("version_number")
  /// Sanitised HTML with variable tokens as <span data-hms-var="…">.
  contentHtml   String   @map("content_html")
  /// Paper size, margins, header/footer repeat, repeating-block columns.
  settings      Json
  publishedAt   DateTime @map("published_at")
  publishedById String?  @map("published_by_id") @db.Uuid

  template DocumentTemplate  @relation(fields: [templateId], references: [id], onDelete: Cascade)
  renders  InvoiceDocument[]

  @@unique([templateId, versionNumber])
  @@map("document_template_versions")
}

enum InvoiceDocumentStatus { PENDING  READY  FAILED }

/// One rendered invoice PDF. `renderedData` is the resolved variable payload
/// captured at issue time — the document is reproducible from this row alone,
/// so a later template edit, tariff reprice, or patient rename cannot rewrite
/// a receipt that is already in a patient's hands.
model InvoiceDocument {
  id                String               @id @default(uuid()) @db.Uuid
  invoiceId         String               @map("invoice_id") @db.Uuid
  templateVersionId String               @map("template_version_id") @db.Uuid
  renderedData      Json                 @map("rendered_data")
  status            InvoiceDocumentStatus @default(PENDING)
  storageKey        String?              @map("storage_key")
  /// SHA-256 of the PDF bytes. Two downloads must match.
  checksum          String?
  sizeBytes         Int?                 @map("size_bytes")
  pageCount         Int?                 @map("page_count")
  renderWarnings    Json                 @default("[]") @map("render_warnings")
  renderError       String?              @map("render_error")
  renderedAt        DateTime?            @map("rendered_at")

  invoice         Invoice                @relation(fields: [invoiceId], references: [id], onDelete: Restrict)
  templateVersion DocumentTemplateVersion @relation(fields: [templateVersionId], references: [id], onDelete: Restric
```

```
  @@index([invoiceId, createdAt])
  @@map("invoice_documents")
}
```

API surface & RBAC

METHOD	PATH	PERMISSION	NOTES
`GET`	`/api/v1/clinic-profile`	`clinic-profile.read:any`	Single row; 404 until configured
`PATCH`	`/api/v1/clinic-profile`	`clinic-profile.write:any`	Audited UPDATE
`POST`	`/api/v1/clinic-profile/logo-upload-url`	`clinic-profile.write:any`	Presigned PUT, image allowlist, re-encode at confirm
`GET`	`/api/v1/document-templates`	`document-template.read:any`	Filter by `kind`
`POST`	`/api/v1/document-templates`	`document-template.write:any`	—
`PATCH`	`/api/v1/document-templates/:id`	`document-template.write:any`	Sanitises `contentHtml` server-side, always
`POST`	`/api/v1/document-templates/:id/publish`	`document-template.write:any`	Creates an immutable version row
`POST`	`/api/v1/document-templates/:id/set-default`	`document-template.write:any`	Transactional flag swap
`POST`	`/api/v1/document-templates/:id/preview`	`document-template.write:any`	Fixture data; persists nothing
`GET`	`/api/v1/document-templates/variables`	`document-template.read:any`	The registry: token, label, type, sample
`POST`	`/api/v1/invoices/:id/document`	`invoice.write:any`	Idempotent per invoice + template version
`GET`	`/api/v1/invoices/:id/document/download`	`invoice.read:any`	Presigned GET, attachment disposition, type pinned

New permission keys, seeded in `prisma/seed.sql`: `clinic-profile.read:any` (ADMIN, DOCTOR, PHARMACIST), `clinic-profile.write:any`, `document-template.read:any` and `document-template.write:any` (ADMIN). Invoice document read and write reuse the existing `invoice.read:any` / `invoice.write:any` grants — a document is part of the invoice, not a separate resource with a separate reach.

Rendering architecture — the decision

The template is authored as HTML, so the renderer must consume HTML. Four options were weighed.

OPTION	FIDELITY	OPS COST	SECURITY POSTURE
Gotenberg sidecar (Chromium)	Exact	One container in compose + deploy	Chromium isolated from the API process; can be network-denied
Puppeteer inside the API image	Exact	API image grows ~300 MB	A renderer bug is a bug in the process holding the DB connection
`@react-pdf/renderer` / `pdfmake`	Poor — needs a translation layer from editor HTML	Low	Good
`wkhtmltopdf`	Dated engine, weak Grid/Flex	Low	Unmaintained upstream

RECOMMENDATION · PROPOSE AS D-026

A **Gotenberg sidecar**, reached over the internal network only. The API POSTs self-contained HTML — every asset already inlined as a `data:` URI — and receives PDF bytes. The container runs with outbound network access denied, so a template that somehow retains a remote reference fetches nothing. This is the largest infrastructure decision in the PRD and should be recorded in `docs/post-mvp/decisions.md` when accepted.

Variable registry (initial set)

Tokens are stable identifiers; labels are translated in the frontend.

TOKEN	TYPE	SAMPLE
`clinic.name`	text	Klinik Sehat Bersama
`clinic.legalName`	text	PT Sehat Bersama Indonesia
`clinic.address`	text	Jl. Merdeka No. 12, Bandung
`clinic.phone` · `clinic.email`	text	(022) 1234567 · halo@kliniksehat.id
`clinic.licenseNumber`	text	440/1234/DPMPTSP
`clinic.logo`	image	embedded
`invoice.number`	text	INV-20260830-0007
`invoice.issuedAt`	date	30 Agustus 2026
`invoice.status`	enum	PAID
`invoice.total`	money	Rp 275.000
`invoice.totalInWords`	text	dua ratus tujuh puluh lima ribu rupiah
`invoice.qrVerify`	image	QR of the verification URL
`patient.fullName`	text	Siti Rahmawati
`patient.mrn`	text	RM-000142
`patient.dateOfBirth` · `patient.sex`	date · enum	4 Februari 1988 · Perempuan
`patient.nikMasked`	text	••••••••••••3271
`encounter.date` · `encounter.doctorName`	date · text	30 Agustus 2026 · dr. Andi Prasetyo, Sp.PD
`admission.roomLabel` · `admission.nights`	text · number	Melati 2A · 3
`payment.method` · `payment.reference`	enum · text	QRIS · QR-88213771
`payment.cashierName`	text	Rina Kartika
`items`	block	repeating rows over `InvoiceItem`

DELIBERATE OMISSION

`patient.nikMasked` is the only identifier token. The plaintext NIK is encrypted at rest and gated behind `patient.read-identifier` — putting it on a receipt the patient carries out of the building is not a layout choice anyone should be able to make in a WYSIWYG editor.

Edge cases & failure modes

CASE	BEHAVIOUR
Renderer sidecar down	Status FAILED with the reason; the UI offers *Retry*. Recording payment is never blocked by a render failure.
Two cashiers hit Download at once on first render	Unique index on `(invoiceId, templateVersionId)`; the loser reads the winner's row.
No published template exists	Fall back to a built-in system template shipped with the API; surface a setup banner to admins.
Template references a removed variable	Renders empty + a warning; publishing shows a blocking validation error listing unknown tokens.
Logo missing from the bucket	Renders without it + a warning; never a failed PDF.
Invoice voided after its PDF was generated	A new watermarked document is rendered; the pre-void document is retained, never deleted — it may already be in a patient's hands.

EPIC 2 OF 5 · 42 POINTS

Patient clinical documents

> Every file that belongs to a patient lives against that patient's record, and the attending doctor can open it without leaving the encounter.

Functional requirements

ID	PRI	REQUIREMENT
FR-E2-01	MUST	Staff can upload a document against a patient with a title, a category, an optional document date and optional notes. A **doctor may upload directly** — but only to a patient assigned to them, never an arbitrary one.
FR-E2-02	MUST	Categories are an explicit enum: lab result, radiology, external medical record, referral letter, consent form, discharge summary, medical certificate, insurance / payer, identity, other.
FR-E2-03	MUST	A document may optionally link to one `Encounter` **or** one `Admission` — the visit it arose from. Unlinked documents belong to the patient generally.
FR-E2-04	MUST	The patient detail page lists documents newest-first, filterable by category and date range, with the linked visit shown.
FR-E2-05	MUST	The encounter workspace shows a **Documents** panel: this visit's documents first, then the patient's history, with a count badge.
FR-E2-06	MUST	A doctor may read a patient's documents when the patient is assigned to them **or** when they are attending an encounter for that patient — the same definition of OWN that `encounter.read:own` already uses.
FR-E2-07	MUST	Every read — list, metadata, download — writes an `AuditLog` row with actor, patient, document and access context. The regulatory question is “who looked”.
FR-E2-08	MUST	Downloads are short-lived presigned GETs with attachment disposition and the validated content type pinned. Nothing renders in the app or API origin.
FR-E2-09	MUST	Accepted types are PDF, JPEG, PNG and WebP. **Every accepted image is decoded and re-encoded through `sharp` before storage** — this strips EXIF/GPS from a phone photo of a lab sheet and destroys polyglot payloads.
FR-E2-10	MUST	The per-surface size cap is 20 MiB, configurable, signed into the presigned URL — a scanned multi-page radiology report does not fit in the current 5 MiB default.
FR-E2-11	MUST	Deletion is soft, requires a reason, and is admin-only. Clinical files fall under the 25-year RME retention floor; nothing here hard-deletes.
FR-E2-12	MUST	Documents are **not** ingested into the retrieval corpus. `PATIENT_CLINICAL` sets `ingestStatus = NOT_APPLICABLE`, so a patient's lab result can never surface as a chatbot citation.
FR-E2-13	SHOULD	A `releasedToPatient` flag controls portal visibility, defaulting to false. **Release is also the trigger for delivery**: the clinician's decision is what sends the document to the patient and notifies the attending doctor, in one action.
FR-E2-14	SHOULD	Multi-file upload: staff drop several scans at once and tag them in one pass.
FR-E2-15	COULD	A document date distinct from upload date, so a result from three weeks ago sorts by when it was produced.

User stories

`US-E2-01` Front desk files what the patient brought

`5 pts`

> As a **front-desk officer**, I want to upload the lab result the patient handed me and tag it, so that the doctor sees it before the consultation.

GIVEN the patient detail page **WHEN** the officer chooses a 3 MB PDF, sets category *Lab result* and document date 25 Aug 2026, and confirms **THEN** it appears at the top of the patient's list within 5 s and an audit row records the create.

GIVEN a 25 MiB file **THEN** the UI refuses it before any upload starts, naming the 20 MiB limit.

GIVEN an executable renamed to `.pdf` **THEN** confirm fails on magic-byte validation, the stored object is deleted before the failure is returned, and `DOCUMENT_UPLOAD_REJECTED` is audited.

`US-E2-02` Doctor reads documents inside the encounter

`8 pts`

> As a **doctor**, I want to see the patient's documents while I write the encounter, so that I am not consulting from memory or a phone photo.

GIVEN an open encounter for an assigned patient with 4 documents, 1 uploaded today **WHEN** the doctor opens the Documents panel **THEN** today's document is grouped under *This visit* and the other 3 under *History*, each showing title, category, document date and uploader.

GIVEN a document row **WHEN** the doctor clicks *Download* **THEN** a presigned URL valid for ≤ 5 minutes is minted, the file downloads as an attachment, and an audit row records the read with the encounter id.

GIVEN a patient not assigned to this doctor and with no encounter they attended **WHEN** the doctor requests that patient's documents by id **THEN** the API returns 403 and the body reveals nothing about whether the document exists.

`US-E2-03` Photographed paperwork is safe to store

5 pts

> As a **security reviewer**, I want phone photos re-encoded on the way in, so that a patient's GPS coordinates do not enter the record with their lab sheet.

GIVEN a JPEG carrying EXIF GPS **WHEN** it is confirmed **THEN** the stored object is the re-encoded output and its metadata shows no GPS or camera tags.

GIVEN a JPEG/PHP polyglot **THEN** re-encoding produces an inert image and the original bytes are never stored.

`US-E2-04` A result reaches the patient only after the clinician sees it

3 pts

> As a **doctor**, I want to control when a result becomes visible in the patient portal, so that a patient does not read a frightening number with no one to ask.

GIVEN a newly uploaded document **THEN** `releasedToPatient` is false and the portal does not list it.

GIVEN a doctor toggling *Release to patient* **THEN** it appears in the portal and the release is audited with actor and timestamp.

`US-E2-05` Mis-filed documents can be corrected

3 pts

> As a **clinic admin**, I want to remove a document filed against the wrong patient, so that one person's result is not in another person's record.

GIVEN an admin with `patient-document.write:any` **WHEN** they delete with reason “filed against wrong patient” **THEN** it disappears from every list, `deletedAt` is set, the object is **not** removed from the bucket, and the deletion is audited with the reason.

GIVEN a doctor without the admin grant **THEN** no delete action is offered and the API refuses it.

Data model delta

Extend the existing `Document` model rather than adding a parallel table. The schema docstring already commits to this: “one ingestion pipeline, one embedding space, one S3 layout”.

```
enum DocumentCategory {
  LAB_RESULT  RADIOLOGY  EXTERNAL_MEDICAL_RECORD  REFERRAL_LETTER
  CONSENT_FORM  DISCHARGE_SUMMARY  MEDICAL_CERTIFICATE
  INSURANCE  IDENTITY  OTHER
}

// DocumentPurpose gains:
//   PATIENT_CLINICAL   — never ingested; ingestStatus is NOT_APPLICABLE
//   DOCTOR_CREDENTIAL  — see E3

model Document {
  // … existing columns unchanged …

  /// Set exactly on PATIENT_CLINICAL rows. A CHECK in the migration ties
  /// purpose and owner together so a clinical file cannot exist without the
  /// patient it belongs to.
  patientId         String?           @map("patient_id") @db.Uuid
  /// The visit this document arose from. At most one of the two is set.
  encounterId       String?           @map("encounter_id") @db.Uuid
  admissionId       String?           @map("admission_id") @db.Uuid
  category          DocumentCategory?
  /// When the document was produced, often not when it was uploaded.
  documentDate      DateTime?         @map("document_date") @db.Date
  notes             String?
  /// False until a clinician releases the result to the patient portal.
  releasedToPatient Boolean           @default(false) @map("released_to_patient")
  releasedAt        DateTime?         @map("released_at")
  releasedById      String?           @map("released_by_id") @db.Uuid
  /// Required on soft delete of a clinical document.
  deleteReason      String?           @map("delete_reason")

  patient   PatientProfile? @relation(fields: [patientId],   references: [id], onDelete: Restrict)
  encounter Encounter?      @relation(fields: [encounterId], references: [id], onDelete: Restrict)
  admission Admission?      @relation(fields: [admissionId], references: [id], onDelete: Restrict)

  @@index([patientId, documentDate])
  @@index([encounterId])
  @@index([purpose, category])
}
```

`onDelete: Restrict` on the patient and visit links is deliberate and matches the existing clinical-FK convention: losing a patient row must never quietly orphan their documents.

API surface & RBAC

METHOD	PATH	PERMISSION
`POST`	`/api/v1/patients/:patientId/documents/upload-url`	`patient-document.write:any`
`POST`	`/api/v1/patients/:patientId/documents`	`patient-document.write:any`
`GET`	`/api/v1/patients/:patientId/documents`	`patient-document.read:own`
`GET`	`/api/v1/patient-documents/:id/download`	`patient-document.read:own`
`PATCH`	`/api/v1/patient-documents/:id`	`patient-document.write:any`
`POST`	`/api/v1/patient-documents/:id/release`	`patient-document.release:own`
`DELETE`	`/api/v1/patient-documents/:id`	`patient-document.write:any`
`GET`	`/api/v1/encounters/:encounterId/documents`	`patient-document.read:own`
`GET`	`/api/v1/portal/me/documents`	`patient-document.read:own`

PERMISSION KEY	ADMIN	DOCTOR	PATIENT	PHARMACIST
`patient-document.read:any`	✔	—	—	—
`patient-document.read:own`	—	✔	✔	—
`patient-document.write:any`	✔	—	—	—
`patient-document.write:own`	—	✔	—	—
`patient-document.release:own`	—	✔	—	—
`patient-document.delete:any`	✔	—	—	—

`read:own` resolves per role in the ability factory: for **doctors**, *own* means assigned patients plus patients whose encounters they attended; for **patients**, their own record and only documents where `releasedToPatient` is true. Pharmacists get nothing — dispensing needs the prescription, not the radiology report.

> **WRITE IS NARROWER THAN READ, DELIBERATELY**
>
> A doctor may write only to patients **assigned** to them — the `DoctorPatient` relationship — not to a patient merely seen once in an encounter. Reading a past visit is clinical necessity; writing into someone's permanent record is not. There is **no break-glass path**: a doctor with no assignment gets no access, with or without a justification prompt. Both confirmed by the product owner.

UX placement

- `app/admin/patients/[id]` — new *Documents* tab; server component composes, client component owns the upload dialog and table.

- `app/doctor/encounters/[id]` — new *Documents* panel, collapsed by default with a count badge, beside vitals and diagnoses.
- New feature folder `components/client/patient-documents/`, one component per file; hooks under `lib/patient-documents/` wrapping the Orval-generated client. No hand-written HTTP.
- No inline preview in v1. The download button is the interaction, and the UI says so rather than implying a viewer is coming.

Edge cases & failure modes

CASE	BEHAVIOUR
Upload confirmed twice (client retry)	Unique `storageKey` returns 409; no second row, no second file.
Presigned URL minted but never used	No row is written; a nightly job reports orphan objects for review — no auto-delete.
Doctor's assignment removed mid-session	The next request re-evaluates the ability and returns 403; nothing is cached client-side.
Patient merged into another record (future)	Documents follow the surviving `patientId` — a stated dependency for any future merge feature.
Encounter soft-deleted	`Restrict` prevents it; the document must be unlinked first.
Corrupt PDF that `pdf-parse` cannot read	Irrelevant — clinical documents are never ingested.

EPIC 3 OF 5 · 29 POINTS

Doctor personal document vault

> A doctor gets private storage inside HMS for their own professional paperwork — visible to them and to nobody else, by construction rather than by permission — and the clinic's licence-expiry obligation is met without anyone ever opening one of those files.

The correction that shapes this epic

An earlier draft of this PRD modelled doctor documents as a **credential file the clinic reviews**: the doctor uploads an STR, an admin verifies it, an admin works an expiry queue over the scans. That is wrong for this product.

A doctor document is the doctor's own. No admin, no super-admin, and no other role can view it. There is no approval on it, because there is nobody to approve it — the only person who ever sees the file is the person who uploaded it.

> THE REPO ALREADY BUILDS ONE THING THIS WAY
>
> The seed file's comment on the personal knowledge base is the governing precedent: "`OWN` only: a personal knowledge base is private to its owner and is filtered by `ownerId` in the repository query, so another doctor's documents are not in the candidate set rather than merely unlikely to rank. **No `ANY` grant for `DOCTOR` at any point** — that would make every personal corpus readable by every clinician."
>
> E3 extends that guarantee to storage. Privacy here is not a permission that could be granted later: **there is no route that takes another user's id**, ownership is derived from the authenticated actor and never accepted from a request, and no `read:any` key exists for this surface to be granted at all.

Two things that share a subject and nothing else

Splitting these is the whole design. They are described together only so nobody re-merges them later.

	THE VAULT	LICENCE EXPIRY TRACKING
Owns it	The doctor	The clinic
Data	Uploaded files — scans, certificates, CV, personal reference material	`DoctorLicense` rows: type, `licenseNumber`, `issuedAt`, `expiresAt` — structured fields that already exist and are already admin-managed
Who can read	**Only the owning doctor**	Admins, as they do today
Involves a document	Yes	**No.** Not one file is read, and none needs to exist
Purpose	The doctor's own filing cabinet	The clinic's compliance obligation

The clinic's real need — *"do not let a doctor practise on a lapsed SIP"* — is a question about **a number and a date**, not about a scan. `DoctorLicense` already carries both and is already indexed on `expiresAt`; its schema comment already says clinics "must track SIP expiry for licensing audits". So the compliance feature is built on the structured record the admin can already see, and it neither reads nor requires the doctor's private file. That is why this epic can give the doctor total privacy and still close the compliance gap.

What a doctor keeps here

The open question in the original brief was "what kind of document". Research into Indonesian practice and standard credentialing checklists gives the categories a doctor's own file actually holds. They are offered as a category list for the doctor's own filing — **not** as a checklist anyone audits them against.

CATEGORY	TYPICALLY HOLDS
Registration & licence	STR (Surat Tanda Registrasi), SIP (Surat Izin Praktik) — one per practice location
Education	Ijazah, transcripts, specialist and sub-specialist certificates
Competence	Sertifikat Kompetensi (Serkom) from the kolegium, IDI branch recommendation
Continuing education	CME / P2KB certificates accumulated over a cycle
Insurance	Professional indemnity / malpractice policy and coverage history
Employment	Contracts, SK, payer credentialing correspondence
Identity & tax	KTP, NPWP, *surat keterangan sehat*
Curriculum vitae	The gap-free dated CV credentialing bodies ask for
Personal reference	Anything else the doctor wants stored and not embedded

Functional requirements — the vault

ID	PRI	REQUIREMENT
FR-E3-01	MUST	A doctor can upload, list, download, rename, re-categorise and delete documents in their own vault.
FR-E3-02	MUST	**Private by default and by construction.** Ownership is derived from the authenticated actor and never accepted from a request. **No route anywhere in the API accepts another user's id.** A document not owned by the caller is reachable only through a live share its owner created — never by naming its owner, and never by browsing.
FR-E3-03	MUST	**No `read:any` or `write:any` key exists for this surface**, for any role, including ADMIN. There is no permission to grant that would let someone browse another person's vault — the only way in is a share the owner chose to create.
FR-E3-04	MUST	Documents carry an optional category, reference number, and issue and expiry dates. Every field is the doctor's own filing metadata; nothing validates it against an external source.
FR-E3-05	MUST	**Never ingested, never sent to any AI provider.** Stored with a purpose whose resting `ingestStatus` is NOT_APPLICABLE — what the schema already describes as "stored and served but never embedded". No chunk is created and no bytes reach an AI vendor.
FR-E3-06	MUST	The vault is a **separate surface from the personal knowledge base**, with its own route group and page. The knowledge base is embedded and its chunks are sent to the AI provider; the vault is not. Merging them would silently send a doctor's KTP to a vendor.
FR-E3-07	MUST	Accepted types are PDF, JPEG, PNG, WebP, with the same re-encode-on-upload, magic-byte validation and 20 MiB cap as E2.
FR-E3-08	MUST	Where a document carries an expiry date, **the owner** is reminded at 60 and 30 days and on expiry. Nobody else is notified — the reminder is a service to the owner, not a report to the clinic.
FR-E3-09	MUST	The owner can hard-delete their own document; deletion removes the stored object and every share on it. These are personal records, **not clinical records**: the 25-year RME retention floor does not apply and must not be applied here.
FR-E3-10	MUST	Reads and writes are audited with the actor, and — where the reader is not the owner — with the share that authorised it.
FR-E3-11	MUST	The existing `document.read:own` / `document.write:own` grants continue to mean *personal knowledge base*; the vault is scoped by purpose within the same owner check, so widening one never widens the other.
FR-E3-12	SHOULD	The owner can **export their whole vault** as a zip, so leaving the clinic does not mean leaving their paperwork behind.

Sharing — private by default, granted by the owner

This is the model the product owner asked for, and it is the one a published artifact uses: **the thing is private until its owner hands someone a key, the key is to one thing and one person, and the owner can take it back and see who used it.** Nothing about the document changes when it is shared; what changes is that a second, named person now has a provable relationship to it.

WHY THIS NEEDS NO NEW PERMISSION

The seed file's comment on `encounter.read:own` says: "`OWN` for a doctor means the encounters they attended plus those of patients actively assigned to them — a clinician needs the previous visits to read the current one, and **the assignment is what says the relationship exists**."

`OWN` in this system has never meant strict ownership; it has meant *a relationship the server can prove*. An explicit, owner-created, revocable share is exactly such a relationship. So sharing needs **no new read permission and no new scope** — `personal-document.read:own` resolves in the ability factory to *owned by me, or shared with me by its owner and still live*. `ANY` still does not exist, so nobody can browse; they can only open what they were handed.

ID	PRI	REQUIREMENT
FR-E3-13	MUST	The owner can grant read access to one of their documents to one or more **named users**. Only the owner can initiate a share; there is no request, prompt, or ask-for-access flow anyone else can start.
FR-E3-14	MUST	A recipient can **view and download, and nothing else**: no edit, no delete, no re-share, and no visibility into any other document in that vault. A share is a key to one door, not to the building.
FR-E3-15	MUST	The owner can revoke any share at any time. Revocation takes effect on the next request — there is no window in which a revoked recipient can still fetch the file.
FR-E3-16	MUST	The owner sees, per document, **who it is shared with, when each recipient last opened it, and how many times**. Being able to watch the door is what makes people willing to open it.
FR-E3-17	MUST	A recipient sees a **Shared with me** list containing only what has been shared with them. It is not a view onto anyone's vault and shows nothing about what else that vault holds.
FR-E3-18	MUST	Every grant, revocation and non-owner access is audited with owner, recipient, document and the authorising share.
FR-E3-19	MUST	A recipient is notified when a document is shared with them, and the owner is notified the first time each recipient opens it.
FR-E3-20	SHOULD	A share may carry an **optional expiry**, so "share this for the accreditation survey" does not quietly become permanent. The owner is reminded about shares older than a configurable age that have no expiry set.
FR-E3-21	SHOULD	Multi-select sharing: the owner can share several chosen documents in one action, creating one share row per document. There is deliberately **no share-my-whole-vault switch** — every share names a document.
FR-E3-22	COULD	Client-side encryption of vault objects under a doctor-held key, so not even an operator with bucket access can read them. Sharing would then exchange a wrapped key rather than a permission row.

WHAT REVOCATION CAN AND CANNOT DO

Revoking stops every future fetch. It does **not** recall a copy already downloaded — the same limitation the E4 delivery analysis states about a WhatsApp attachment, and for the same reason. The UI says this plainly at the point of sharing rather than implying a recall that does not exist.

User stories

US-E3-01 Doctor files their own paperwork

5 pts

> As a **doctor**, I want somewhere in HMS to keep my STR, my certificates and my CV, so that they are with the rest of my working life instead of on my phone.

GIVEN the doctor's *My Documents* page **WHEN** they upload an STR scan, category *Registration & licence*, expiry 12 Mar 2029 **THEN** it appears in their vault immediately, `ingestStatus` is NOT_APPLICABLE, and no chunk row is created.

GIVEN a 25 MiB file **THEN** the UI refuses it before any upload starts, naming the 20 MiB limit.

GIVEN a JPEG carrying EXIF GPS **THEN** the stored object is the `sharp` re-encoded output with no GPS or camera tags.

US-E3-02 Nobody sees it unless the owner says so

5 pts

> As a **doctor**, I want certainty that no admin can open my KTP unless I hand it to them, so that I am willing to put it here at all.

GIVEN an ADMIN with every permission the seed grants **WHEN** they call any vault route **THEN** no route accepts another user's id, and the only vault they can list is their own.

GIVEN a document with no share **WHEN** anyone other than the owner requests it by id **THEN** the repository query returns nothing and the response is a 404 that reveals nothing about existence.

GIVEN the RBAC seed **THEN** no `read:any` key exists for this surface, and a guard test asserts that no role holds one.

`US-E3-03` The vault never feeds the chatbot

3 pts

> As a **doctor**, I want my personal documents kept out of the AI pipeline, so that uploading my tax file does not send it to a vendor.

GIVEN a vault document **THEN** its `ingestStatus` is NOT_APPLICABLE, no `DocumentChunk` is ever created, and the retrieval query cannot return it.

GIVEN the doctor's *My Documents* page **THEN** it states plainly that these files are private and are not used by the assistant — the inverse of the notice the knowledge base carries.

`US-E3-04` The doctor is reminded, and only the doctor

3 pts

> As a **doctor**, I want to be told before my SIP expires, so that I renew it in time — without my clinic being told first.

GIVEN a vault document with an expiry 30 days out **WHEN** the daily job runs **THEN** only the owning doctor receives a bell notification, and no admin notification is created.

GIVEN the job runs twice in a day **THEN** no duplicate notification is produced for the same document and threshold.

US-E3-05 Doctor shares one document for a survey

5 pts

> As a **doctor**, I want to hand my STR to our clinic admin for the accreditation survey, so that they have what they need without me sending it over WhatsApp.

GIVEN an STR in the doctor's vault **WHEN** they share it with a named admin and set the share to expire in 30 days **THEN** the admin is notified, the document appears in that admin's *Shared with me* list, and nothing else in the vault becomes visible to them.

GIVEN that admin **WHEN** they open the document **THEN** they can view and download it, and there is no rename, delete or share action available to them anywhere in the UI or the API.

GIVEN a second admin who was not named **WHEN** they request the same document by id **THEN** they get a 404 — being an admin grants nothing here.

GIVEN the share's expiry passes **THEN** the next fetch is refused and the document leaves that admin's list without the owner doing anything.

US-E3-06 Owner watches the door and can close it

5 pts

> As a **doctor**, I want to see who opened what I shared and take it back, so that sharing feels like lending rather than giving away.

GIVEN a document shared with two admins, one of whom has opened it twice **WHEN** the owner opens the document's *Sharing* panel **THEN** they see both recipients, the last-opened time and open count for each, and a *Revoke* action per recipient.

GIVEN the owner revokes one share **THEN** the next request from that recipient is refused, the document leaves their *Shared with me* list, and the revocation is audited.

GIVEN a recipient opens a shared document for the first time **THEN** the owner receives a notification.

GIVEN a share created 90 days ago with no expiry **THEN** the owner is reminded that it is still open.

`US-E3-07` **The doctor can take it with them** 3 pts

> As a **doctor leaving a clinic**, I want to export and delete my own documents, so that my personal records are not stranded in someone else's system.

GIVEN a vault with 12 documents **WHEN** the doctor exports **THEN** they receive a zip of all of them with their metadata.

GIVEN a document the doctor deletes **THEN** the row, the stored object and every share on it are removed, and no retention rule blocks it.

Data model delta

Small, because the shape already exists. `Document` needs a purpose value of the kind it already has, a few filing fields, and one join table for shares.

```
// DocumentPurpose gains:
//   PERSONAL_DOCUMENT — the vault. Stored, served to its owner and to anyone
//   the owner shared it with, never ingested.

enum PersonalDocumentCategory {
  REGISTRATION_LICENCE  EDUCATION  COMPETENCE  CONTINUING_EDUCATION
  INSURANCE  EMPLOYMENT  IDENTITY_TAX  CURRICULUM_VITAE
  PERSONAL_REFERENCE  OTHER
}

model Document {
  // … existing + E2 columns …

  /// Filing metadata on a PERSONAL_DOCUMENT row. Every field is the owner's
  /// own note to themselves — nothing here is validated against an external
  /// register, because nobody reads it who the owner did not invite.
  personalCategory PersonalDocumentCategory? @map("personal_category")
  referenceNumber  String?                   @map("reference_number")
  issuedAt         DateTime?                 @map("issued_at") @db.Date
  expiresAt        DateTime?                 @map("expires_at") @db.Date

  shares PersonalDocumentShare[]

  @@index([ownerId, personalCategory])
  @@index([ownerId, expiresAt])
}

/// One person's access to one document, created only by that document's owner.
///
/// This row *is* the relationship that makes `OWN` resolve for a non-owner, the
/// same way `DoctorPatient` is what makes `encounter.read:own` resolve for a
/// doctor. There is no share-all row and no role target in v1: a share names a
/// document and a person, so revoking one never has to reason about a set.
///
```

```
/// `lastAccessedAt` / `accessCount` are denormalised here rather than derived
/// from `AuditLog` because the owner-facing panel reads them on every render
/// and the owner has no permission to query the audit log. The audit log
/// remains the forensic record; these two columns are the product surface.
model PersonalDocumentShare {
  id             String    @id @default(uuid()) @db.Uuid
  documentId     String    @map("document_id") @db.Uuid
  /// Who was given the key.
  granteeId      String    @map("grantee_id") @db.Uuid
  /// Always the document's owner, re-checked in the service on every write —
  /// a share created by anyone else is the one bug that would undo the epic.
  grantedById    String    @map("granted_by_id") @db.Uuid
  expiresAt      DateTime? @map("expires_at")
  revokedAt      DateTime? @map("revoked_at")
  lastAccessedAt DateTime? @map("last_accessed_at")
  accessCount    Int       @default(0) @map("access_count")

  document  Document @relation(fields: [documentId],  references: [id], onDelete: Cascade)
  grantee   User     @relation("PersonalDocumentGrantee", fields: [granteeId],   references: [id], onDelete: Cascade
  grantedBy User     @relation("PersonalDocumentGrantor", fields: [grantedById], references: [id], onDelete: Cascade

  /// One live share per (document, person). Re-sharing after a revoke updates
  /// this row rather than accumulating history the owner has to read past.
  @@unique([documentId, granteeId])
  @@index([granteeId, revokedAt])
  @@map("personal_document_shares")
}

// NotificationType gains:
//   PERSONAL_DOCUMENT_EXPIRING   PERSONAL_DOCUMENT_EXPIRED   (owner only)
//   PERSONAL_DOCUMENT_SHARED     (recipient)
//   PERSONAL_DOCUMENT_OPENED     (owner, first open per recipient)
//   LICENCE_EXPIRING             LICENCE_EXPIRED             (clinic side)
```

API surface & RBAC

Owner routes live under `me/`. The shared-document routes address a **document id**, never a user — which is what keeps `FR-E3-02` structural: there is no request shape anywhere in the API that names whose vault you want.

METHOD	PATH	PERMISSION
`POST`	`/api/v1/me/personal-documents/upload-url`	`personal-document.write:own`
`POST`	`/api/v1/me/personal-documents`	`personal-document.write:own`
`GET`	`/api/v1/me/personal-documents`	`personal-document.read:own`
`GET`	`/api/v1/me/personal-documents/:id/download`	`personal-document.read:own`
`PATCH` · `DELETE`	`/api/v1/me/personal-documents/:id`	`personal-document.write:own`
`GET`	`/api/v1/me/personal-documents/export`	`personal-document.read:own`
`GET` · `POST`	`/api/v1/me/personal-documents/:id/shares`	`personal-document.share:own`
`DELETE`	`/api/v1/me/personal-documents/:id/shares/:shareId`	`personal-document.share:own`
`GET`	`/api/v1/shared-with-me/documents`	`personal-document.read:own`
`GET`	`/api/v1/shared-with-me/documents/:id/download`	`personal-document.read:own`

PERMISSION KEY	ADMIN	DOCTOR	NOTES
`personal-document.read:own`	✔[1]	✔	Own vault, plus documents shared with them. **`ANY` does not exist as a key**
`personal-document.write:own`	✔[1]	✔	Own vault only. Never applies to a shared document
`personal-document.share:own`	✔[1]	✔	Grant and revoke on **own** documents only

[1] An admin holds these for **their own** vault, exactly as a doctor does — an admin is also a person with a contract and a KTP. None of them grants anything over anyone else's documents.

`share:own` is a separate key from `write:own` for the same reason `invoice.deliver:any` is separate from `invoice.write:any`: handing a document to someone else is a different act from editing it, and a deployment that wants to disable sharing entirely should be able to do that without disabling the vault.

Licence expiry tracking — clinic-side, no documents involved

This is the compliance half, and it touches no file. It exists so the clinic's obligation never depends on a doctor choosing to share anything.

ID	PRI	REQUIREMENT
FR-E3-33	MUST	An admin dashboard lists `DoctorLicense` rows expiring in 30 / 60 / 90 days and those already expired, sorted by urgency, from the structured fields admins already manage.
FR-E3-34	MUST	Expiry produces bell notifications to admins at 60 and 30 days and on expiry, deduplicated per licence and threshold.
FR-E3-35	MUST	The dashboard shows the licence **number and dates only**. It never links to, references, or reveals the existence of a document in anyone's vault — including one shared with the viewer.
FR-E3-36	SHOULD	A doctor with an expired STR or SIP is flagged when scheduling: the appointment-session UI shows a warning to the scheduler.
FR-E3-37	COULD	Hard-block scheduling for a doctor with an expired SIP, behind a clinic-level setting defaulting to off.

US-E3-08 Nothing expires unnoticed

5 pts

> As a **clinic admin**, I want to be warned before a doctor's licence lapses, so that no one practises on an expired permit.

- **Given** a `DoctorLicense` of type SIP expiring in 30 days **When** the daily job runs **Then** every admin receives one bell notification and the licence appears in the *Expiring soon* list.
- **Given** an expired SIP **Then** the doctor's row is flagged in the admin directory with the expiry date.
- **Given** the admin opens the expiry dashboard **Then** it shows licence type, number and dates, and **no document, filename, or indication that a scan exists** — even for a document that doctor has shared with them.

US-E3-09 Scheduling sees the risk

3 pts

> As a **scheduler**, I want to know a doctor's practice permit has lapsed before I book patients into their session.

- **Given** a doctor with an expired SIP **When** the scheduler opens their appointment sessions **Then** a warning banner names the lapsed licence and its expiry date; booking still proceeds in v1 (`FR-E3-36` is a warning; `FR-E3-37` is the optional hard block).

Offboarding — what happens to a vault when a doctor leaves

The product owner's rule, in three parts:

1. **Shared documents survive.** Anything the doctor shared stays readable by the people they shared it with. The clinic keeps the evidence it was given rather than losing an accreditation file the day someone resigns. 2. **Unshared documents stay private and expire after 30 days.** They remain unreadable by anyone — resignation grants nobody access — and are hard-deleted with their stored objects when the window ends. 3. **The doctor is warned and offered the choice:** delete now, or let the window run.

Thirty days is short, and that has consequences

Thirty days is a deliberate, confirmed choice, and it is short for someone's professional paperwork. Two things follow that would otherwise make the promise hollow.

The warning cannot be an in-app notification. A doctor who has resigned may never open the portal again, and a bell nobody sees is not a warning. The offboarding notice goes **by email**, to the address on their user record, on the day the account is deactivated — and again at seven days remaining. It states the date, what will be deleted, what will survive, and how to export.

They need access to act on it. "Delete now or wait" requires being able to log in, and deactivation normally removes that. So deactivating a doctor puts their vault into a **30-day export-only window**: they can sign in, list, download, export and delete their own documents, and do nothing else in the system. The window is the mechanism that makes the choice real; without it the rule is just a countdown to deletion.

The offboarding state

Offboarding is **a super admin action, and it is not deactivation.** The two must stay separate, because they answer different situations:

	OFFBOARD	DEACTIVATE (EXISTS TODAY)
Situation	A doctor resigns and leaves on good terms	A security incident, a dismissal for cause, a suspected compromise
Access after	30 days, vault only	**None, immediately**
Vault documents	Expire in 30 days; the doctor may take or delete them first	Expire in 30 days; the doctor never gets the window

A clinic that needs someone out *now* still deactivates, and nothing about this epic slows that down. Collapsing the two would mean every dismissal handed the dismissed person a month of access.

What an offboarded doctor can do. Exactly one thing: their own vault. Every other capability is gone the moment the super admin offboards them — no patients, no encounters, no appointments, no prescriptions, no knowledge base, no chat, no directory. Signing in lands them on their documents and nowhere else.

> **ONE INTERPRETATION STATED PLAINLY**
>
> The instruction was *"no permission to all feature except for document deletion."* This is built as **view, download, export and delete of their own vault** — deletion plus the ability to take a copy first. A delete-only window would make the choice meaningless: the doctor could only destroy their own STR and CV or watch them expire, with no way to keep them. If delete-only is genuinely intended, it is a one-line change to `FR-E3-23` and everything else here holds.

How the reduced state is enforced

Three mechanisms, and the reasoning for each matters more than the fields.

`offboardedAt` on the user, not a role. The reduced capability set is a **hard-coded branch in the ability factory**, not a seeded role and not a permission grant. A role can be edited in the portal and quietly widened; a code branch cannot. This is the same reasoning that made `personal-document.read:any` a key that does not exist rather than a key nobody holds — the strongest guarantee is the one that is not configurable.

The login path branches on it, the way it already does for service accounts. `User.isSystem` carries the note that *"what makes it a non-identity is that the login path refuses it outright"*; `offboardedAt` uses the same hook to refuse a sign-in once the window has closed, rather than relying on a scheduled job having run.

Every session is revoked at offboarding. D-022 accepts a staleness window for permission changes because a role edit is rarely urgent. This one is: a doctor holding a live session would keep full access until their token expired, and the permissions the web app renders from come from the packed session hint, not a fresh lookup. So offboarding revokes the refresh-token family and forces re-auth, audited as `SESSION_REVOKED_ALL` — the reduced set takes effect on the next request, not on the next token refresh.

ID	PRI	REQUIREMENT
FR–E3–23	MUST	Offboarding is a distinct super-admin action, separate from deactivation. It sets `offboardedAt` and opens a 30-day window in which the doctor may **view, download, export and delete their own vault documents, and do nothing else in the system**.
FR–E3–24	MUST	The reduced capability set is a **hard-coded branch in the ability factory**, keyed on `offboardedAt` — not a role, not a permission grant, and not editable from any settings screen.
FR–E3–25	MUST	Offboarding **revokes every session** and forces re-authentication, so the reduced set applies immediately rather than after a token expires.
FR–E3–26	MUST	Once the window closes, the login path refuses the account outright, exactly as it does for a service account.
FR–E3–27	MUST	An offboarding notice is sent **by email** on offboarding and again at 7 days remaining, naming the deletion date, what will be deleted, what will survive, and how to export.
FR–E3–28	MUST	At the end of the window, every **unshared** document is hard-deleted with its stored object; the deletion is audited with a count.
FR–E3–29	MUST	**Shared documents survive** and stay readable by their recipients. The former owner has no access to them once the window closes.
FR–E3–30	MUST	Re-onboarding before the window closes clears `offboardedAt`, cancels the deletion and restores normal access.
FR–E3–31	SHOULD	The super admin sees what will happen before confirming — how many documents are shared, how many will be deleted, and on what date.
FR–E3–32	SHOULD	A doctor who was **deactivated** rather than offboarded can still be sent their export: an admin triggers a one-off export emailed to the doctor, which the admin never sees. Losing your job should not mean losing your own KTP.

Retention here is a **product** rule, not a regulatory one: these are the doctor's personal records, so the 25-year RME floor does not apply and does not constrain the choice.

Edge cases & failure modes

CASE	BEHAVIOUR
A doctor leaves the clinic	**Shares survive; unshared documents stay private and expire.** The clinic keeps evidence it was given; everything unshared stays unreadable and is deleted at the end of the retention window. See the section above.
An admin needs an STR scan for a survey	They ask the doctor, who shares it with an expiry. The evidence reaches the clinic inside the system, with an access log, instead of over WhatsApp.
A recipient forwards the downloaded file	Outside the system's reach, and stated as such at the point of sharing. The access log records that they downloaded it, which is the accountability the product can actually provide.
A share is created and forgotten	Optional expiry plus an owner reminder on old open-ended shares; the sharing panel makes standing shares visible rather than buried.
Recipient is deactivated	Their shares stop resolving with the account; no orphaned access remains.
Owner deletes a shared document	The document, the object and every share go together; recipients see it disappear from *Shared with me*.
A doctor uploads a patient's document by mistake	It is their vault, so it is not exposed — but it is also outside the clinical record and its retention rules. The upload page warns that patient data belongs in the patient's record, mirroring the knowledge base's existing no-patient-data notice.
An operator with bucket access	Can read objects, as they can for every surface. `FR–E3–22` (client-side encryption) is the only real answer and is a COULD.
Licence expiry known to the clinic, scan never shared	Entirely normal and not an error state. The dashboard never notices, because it never looks.

EPIC 4 OF 5 · 39 POINTS

Invoice delivery over WhatsApp and email

> The patient gets their receipt on the channel they already use, without the clinic printing it, and the clinic can prove what was sent, to whom, and when.

What already exists — and what genuinely does not

This epic is unusual in this PRD: both transports are **already in production**, so the work is narrower than it looks — but the gaps in them are precise.

PIECE	STATE
WhatsApp bridge	**Ships.** `GowaWhatsappAdapter` (`/send/message`) with `WahaWhatsappAdapter` as the tested fallback, selected by `WA_GATEWAY_KIND` . GOWA runs as a private-network container with basic auth and a persisted session volume.
SMTP	**Ships.** `MailService` over nodemailer with a `log` transport for local dev; provider-neutral through six `MAIL_*` variables.
Verified patient phone number	**Ships.** `ChannelPatientLink.verificationStatus = VERIFIED` , proven by an OTP or contact-share challenge.
Send pacing and dispatch	**Ships.** `WhatsappBridgeHttpClient` serialises sends at `WA_GATEWAY_SEND_PACING_MS` ; `OutboundMessageDispatcherService` picks the adapter per channel.
Sending a file over WhatsApp	**Missing.** `WhatsappGatewayService` has exactly one method, `sendText` . Its docstring is explicit that the port stays narrow until “something calls” a wider member — this epic is that call.
Sending an attachment by email	**Missing.** `SendMailRequest` is `{ to, subject, text, html }` . No attachment field exists.
Any delivery record	**Missing.** Nothing persists that an invoice was sent, to which address, or whether it arrived.
Any delivery consent	**Missing.** `PatientProfile.email` is optional free text typed at the counter, with no verification column anywhere.

THE PORT QUESTION

`WhatsappGatewayService` was deliberately kept to one method because “a port is only as portable as its narrowest member”, and D-CS-01 names the official WhatsApp Cloud API as the endgame implementation. Adding `sendDocument` is therefore only acceptable because the Cloud API can satisfy it too — it sends document messages natively. A member the endgame cannot implement would break the hedge against a ban, and must not be added.

The delivery-shape decision — attachment, with a password

An earlier draft recommended a revocable link by email, because an email address is unverified free text and a link can be killed after a misdelivery. The product owner chose differently, and gave the mechanism that makes it work:

> **NOTE**
>
> *"I like attachment, it's easy to access. But for security we can add password to the document. For example, a patient doing a lab test and wants it sent via WA in 3 days — when the time comes the document is sent with the patient's DOB as the password."*

Decision: the attachment is the default on both channels, and every attachment is a password-protected PDF. The password moves the protection from *"can we take it back"* to *"can the wrong recipient open it"*, which is the failure that actually happens: a mistyped digit or address, not a determined attacker.

	PLAIN ATTACHMENT	PASSWORD-PROTECTED ATTACHMENT	REVOCABLE LINK
Patient effort	None	Type a password they already know	Needs data and a browser
Wrong recipient can read it	Yes	**No**	No
Works offline, keeps forever	Yes	Yes	No
After misdelivery	Nothing to do	The file is inert to them	Revoke the token
Audit	"We sent it"	"We sent it"	"...and it was opened at 14:32"

What the password is and is not. The default is the patient's date of birth, in a documented format, because it is something the patient knows without being told and a stranger holding a misdialled number does not. It is **not** a secret in the cryptographic sense — a determined attacker who knows the patient can guess it. It is the right control for misdelivery and the wrong one for a targeted attack, and the PRD says so rather than implying otherwise. The password source is configurable per clinic (`FR-E4-06`) so a clinic wanting something stronger can use one.

Propose as **D-027** in `docs/post-mvp/decisions.md` .

Scheduled delivery

The same answer introduced a second requirement: *"send via WA in 3 days — when the time comes the document will be sent."* Delivery therefore has a **send-at**, not only a send-now. The front desk agrees a date with the patient at the counter, and the outbox worker that already handles retries handles the wait — no new mechanism, one new column and one predicate.

Functional requirements

ID	PRI	REQUIREMENT
FR-E4-01	MUST	From the invoice detail view, a cashier can send the rendered invoice document to the patient over WhatsApp, email, or both, in one action.
FR-E4-02	MUST	A send is refused unless the invoice is `ISSUED` or `PAID` **and** its `InvoiceDocument` is `READY`. A document that failed to render is never "sent as pending".
FR-E4-03	MUST	A WhatsApp send is refused unless the patient has a `ChannelPatientLink` in `VERIFIED` state whose `patientId` is this patient. An unverified or absent link offers the cashier the OTP flow instead of a send.
FR-E4-04	MUST	Delivery over any channel requires a recorded, per-channel **delivery consent** for that patient, captured with the privacy-notice version in force at capture time. No consent, no send — the button is disabled with the reason.
FR-E4-05	MUST	**The attachment is the default on both channels**, and every attachment is a password-protected PDF. Link delivery remains available as a per-channel option for a clinic that prefers it.
FR-E4-06	MUST	The PDF is encrypted with a **user password** before it leaves the system. The default source is the patient's date of birth as `DDMMYYYY`; the source is configurable per clinic. The password is never included in the message that carries the file.
FR-E4-07	MUST	**A patient with no `dateOfBirth` on file cannot receive a password-protected document.** The send is refused with a message naming the missing field and a link to complete the record. Front-desk registration captures it, so this is a data-completeness prompt rather than a common path — but the chat-created drafts the schema deliberately allows have no DOB, and they must be completed before delivery.
FR-E4-08	MUST	The message tells the recipient **what the password is** without disclosing it — "open with your date of birth, DDMMYYYY" — so a patient can open it without a support call and a stranger learns nothing they did not already need.
FR-E4-09	MUST	**Scheduled delivery.** A send can carry a `sendAt` in the future; the outbox worker dispatches it when due, using the same lease, retry and pacing path as an immediate send. Before it fires it can be cancelled or rescheduled.
FR-E4-10	MUST	A scheduled delivery re-checks consent, channel verification and invoice state **at send time, not at scheduling time**. Consent withdrawn or an invoice voided in the intervening days cancels the send.
FR-E4-11	MUST	A link delivery mints a **single-invoice, revocable, expiring token** (default 7 days). Opening it serves the PDF via a short-lived presigned GET; the token is not the storage key and never appears in a bucket URL.
FR-E4-12	MUST	Every attempt writes an `InvoiceDelivery` row: channel, masked destination, shape, status, attempt count, provider message id, error. Status is one of `QUEUED`, `SENT`, `DELIVERED`, `OPENED`, `FAILED`, `REVOKED`.
FR-E4-13	MUST	Sending is asynchronous through a lease-claimed outbox worker with bounded retries and exponential backoff — the same pattern the SATUSEHAT submission worker already uses, so two replicas cannot double-send.
FR-E4-14	MUST	The invoice detail view shows a delivery timeline: what was sent, on which channel, to which masked destination, by whom, with the current status and a retry action.
FR-E4-15	MUST	Message copy is Indonesian-first, names the clinic, the invoice number and the amount, and carries no clinical content — no diagnosis, no procedure names, no medication names. An invoice line description that would reveal a diagnosis is the reason this is a requirement and not a guideline.
FR-E4-16	MUST	A patient can opt out. An inbound `STOP` / `BERHENTI` on WhatsApp revokes delivery consent for that channel, is confirmed in-channel, and is honoured before the next send.

ID	PRI	REQUIREMENT
FR-E4-17	MUST	Invoice sends never starve conversation replies: they enqueue behind interactive traffic on the shared `WA_GATEWAY_SEND_PACING_MS` queue. A daily cap exists as configuration and ships **unset** — see §7.4.4.1.
FR-E4-18	MUST	Every send, open, revoke and opt-out is audited with actor, patient, invoice and channel.
FR-E4-19	SHOULD	Automatic send on payment stays **off**. The front desk asks the patient at the counter — WhatsApp, email, or hard copy — and sends from the portal accordingly. Confirmed by the product owner: the choice is a conversation, not a default.
FR-E4-20	SHOULD	A revoked or superseded invoice (voided, reissued) marks its outstanding delivery links `REVOKED`, so a link in a chat thread stops resolving to a bill that is no longer valid.
FR-E4-21	SHOULD	Email deliveries render from a template sharing the clinic profile (FR-E1-15), so the mail and the PDF carry the same identity.
FR-E4-22	COULD	Bounce handling: an SMTP hard bounce marks the delivery `FAILED` with the reason and flags the address for correction at the next visit.
FR-E4-23	COULD	Patient-side email verification (a one-time confirm link) that upgrades email to attachment-eligible.

Clinical documents go to both ends at once

Delivery was scoped to invoices, on the reasoning that a patient reading a serious result alone, with no clinician present, is a clinical-safety problem. The product owner's answer corrected the premise:

> **NOTE**
>
> *"Sometimes the patient knows first and brings the result to the doctor. But maybe in our system we can integrate both — lab result immediately sent to both ends: patient via WA, and doctor through document."*

That is right, and it is better than either alternative. The patient often receives the physical result first today, so withholding the digital copy protects nobody — it just makes the clinic slower than the lab. And the failure the original concern actually named was not *the patient sees it*, it was *the patient sees it and the doctor does not*. **Dual delivery removes that failure rather than delaying it.**

So a released patient document goes two places in one action:

END	CHANNEL	WHAT ARRIVES
Patient	WhatsApp or email, password-protected attachment, per E4's rules	The document itself
Doctor	The encounter Documents panel (E2) and a bell notification	The document in the patient's record, in clinical context

The clinician's release decision stays the gate. Nothing is delivered on upload; delivery happens on release (FR-E2-13), which is a clinician's call and already exists. The change

is that release now has a destination as well as a visibility flag.

ID	PRI	REQUIREMENT
`FR-E4-24`	**MUST**	Releasing a patient clinical document (E2) can dispatch it to the patient over WhatsApp or email in the same action, under every rule in this epic: consent, verified number, password protection, audit.
`FR-E4-25`	**MUST**	The same release notifies the **attending doctor** in-app, and the document is already in the encounter panel — the doctor is never behind the patient.
`FR-E4-26`	**MUST**	Delivery of a clinical document happens **only on release**, never on upload. A result sitting unreleased is never sent, whatever the front desk does.
`FR-E4-27`	**MUST**	Message copy for a clinical document names the clinic, the document type and the date, and **no result values, diagnosis or interpretation** — the same rule as the invoice caption, for the same reason.
`FR-E4-28`	**SHOULD**	A per-category delivery default, so lab results can dispatch on release while, say, a consent form does not.

On the daily send cap — why it exists, and why it ships off

The product owner asked the fair question: *"why should we limit anyway?"*

The cap is not about our storage or our cost. It is about **the clinic's WhatsApp number**. GOWA drives a WhatsApp Web session, not the official Business API, and a number that suddenly emits a burst of document messages looks more like automation than the booking replies it sends today. The failure it guards against is the number being banned — which takes bookings down with it, not just receipts.

There is no data yet to set a number, and a guessed cap is a support ticket waiting to happen. So: **the cap ships unset**, the metrics in `NFR-OBS-02` record actual daily volume from day one, and a value is chosen in production once the clinic's own traffic is known. The mechanism exists so that turning it on later is configuration, not a release.

User stories

`US-E4-01` Cashier sends the receipt to WhatsApp

`5 pts`

> As a **cashier**, I want to send the invoice to the patient's WhatsApp after they pay, so that they leave with the receipt on their phone and I stop printing.

GIVEN a PAID invoice with a READY document, a VERIFIED WhatsApp link and recorded consent **WHEN** the cashier clicks *Send* → *WhatsApp* **THEN** a delivery row is created QUEUED, the worker sends the PDF as a document message with an Indonesian caption naming the clinic, invoice number and amount, and the timeline shows SENT with the masked number within 30 s.

GIVEN the same invoice already sent once **WHEN** the cashier sends again **THEN** a second delivery row is created — resending is a normal act, not an error — and the timeline shows both attempts.

GIVEN GOWA is disconnected **THEN** the delivery stays QUEUED, retries with backoff, and the UI shows “WhatsApp gateway unavailable” rather than a false success.

`US-E4-02` An unverified number cannot receive a bill

`5 pts`

> As a **records officer**, I want a bill to be un-sendable to a number nobody proved, so that a mistyped digit does not put a patient's charges in a stranger's chat.

GIVEN a patient with no `ChannelPatientLink` **WHEN** the cashier opens *Send*
THEN WhatsApp is disabled with “This patient's number is not verified”, and the cashier is offered the OTP verification flow.

GIVEN a link in UNVERIFIED state **THEN** the same refusal applies — verification state, not the presence of a number, is the gate.

GIVEN a link verified for a *different* patient **THEN** the send is refused and the attempt is audited.

US-E4-03 Email delivers a revocable link

5 pts

> As a **clinic admin**, I want emailed invoices to be a link I can kill, so that a typo in an address is a recoverable mistake.

GIVEN an invoice sent by email **THEN** the mail body contains no PDF and no charge detail — only the clinic identity, the invoice number, and a tokenised link valid for 7 days.

GIVEN the patient opens the link **THEN** the PDF is served with attachment disposition, the delivery moves to OPENED with a timestamp, and no login is required.

GIVEN an admin clicks *Revoke link* **THEN** the token stops resolving within seconds, the status becomes REVOKED, and the revocation is audited.

GIVEN an expired token **THEN** the page says it expired and to contact the clinic — it never reveals whether the invoice exists.

US-E4-04 Delivery failures are visible and retryable

3 pts

> As a **cashier**, I want to see that a send failed, so that I hand over paper instead of assuming it arrived.

GIVEN a delivery that exhausted its retries **THEN** the timeline shows FAILED with a plain-language reason and a *Retry* action, and the invoice is flagged in the day's cashier view as undelivered.

GIVEN a number that is not on WhatsApp **THEN** GOWA's account validation rejects it, the delivery fails immediately rather than after five retries, and the reason says the number is not on WhatsApp.

`US-E4-05` The patient can make it stop

5 pts

> As a **patient**, I want to stop receiving documents on WhatsApp, so that my billing does not keep arriving in a chat I share with family.

GIVEN a patient with delivery consent **WHEN** they send `BERHENTI` to the clinic's number **THEN** consent for WhatsApp is revoked, a confirmation is sent in Indonesian, and the revocation is audited.

GIVEN consent revoked **WHEN** a cashier attempts a WhatsApp send **THEN** it is refused with “The patient opted out of WhatsApp delivery”, and email — if separately consented — is still offered.

`US-E4-06` No clinical content leaves on a billing channel

3 pts

> As a **records officer**, I want the message itself to carry no clinical detail, so that a notification preview on a lock screen is not a disclosure.

GIVEN an invoice whose line items include a procedure description **THEN** the WhatsApp caption and the email subject and body contain the clinic name, invoice number, date and total only — the itemisation exists solely inside the PDF.

GIVEN the email subject line **THEN** it names no diagnosis, procedure or medication.

Data model delta

```
enum DeliveryChannel       { WHATSAPP  EMAIL }
enum DeliveryShape         { ATTACHMENT  LINK }
enum InvoiceDeliveryStatus { QUEUED  SENT  DELIVERED  OPENED  FAILED  REVOKED }

/// Per-patient, per-channel permission to receive documents, with the notice
/// version in force when it was captured. Consent is a fact with a date and a
/// text behind it, not a boolean on the patient row — the same reasoning that
/// gave privacy-notice acceptance its own table.
model PatientDeliveryConsent {
  id              String          @id @default(uuid()) @db.Uuid
  patientId       String          @map("patient_id") @db.Uuid
  channel         DeliveryChannel
  isGranted       Boolean         @default(true) @map("is_granted")
  noticeVersionId String?         @map("notice_version_id") @db.Uuid
  grantedAt       DateTime?       @map("granted_at")
  grantedById     String?         @map("granted_by_id") @db.Uuid
  revokedAt       DateTime?       @map("revoked_at")
```

```
  /// PATIENT_KEYWORD when the patient sent STOP/BERHENTI; STAFF when a clerk
  /// withdrew it at the counter. Two different facts about consent.
  revokedReason   String?         @map("revoked_reason")

  patient       PatientProfile        @relation(fields: [patientId],       references: [id], onDelete: Cascade)
  noticeVersion PrivacyNoticeVersion? @relation(fields: [noticeVersionId], references: [id], onDelete: SetNull)

  @@unique([patientId, channel])
  @@map("patient_delivery_consents")
}

/// One attempt to put one rendered invoice in front of one patient.
///
/// `destinationMasked` is what is stored for display — `0812****7731`,
/// `r***@gmail.com`. The full destination is not duplicated here: it already
/// lives on the link or the patient row, and a delivery log that accumulates
/// plaintext contact details becomes its own disclosure risk.
model InvoiceDelivery {
  id                String               @id @default(uuid()) @db.Uuid
  invoiceId         String               @map("invoice_id") @db.Uuid
  invoiceDocumentId String               @map("invoice_document_id") @db.Uuid
  channel           DeliveryChannel
  shape             DeliveryShape
  destinationMasked String               @map("destination_masked")
  status            InvoiceDeliveryStatus @default(QUEUED)
  attemptCount      Int                  @default(0) @map("attempt_count")
  /// Lease claimed by one worker replica — the SATUSEHAT outbox pattern.
  leasedUntil       DateTime?            @map("leased_until")
  leasedBy          String?              @map("leased_by")
  providerMessageId String?              @map("provider_message_id")
  lastError         String?              @map("last_error")
  sentAt            DateTime?            @map("sent_at")
  openedAt          DateTime?            @map("opened_at")
  requestedById     String?              @map("requested_by_id") @db.Uuid

  invoice  Invoice              @relation(fields: [invoiceId],         references: [id], onDelete: Restrict)
  document InvoiceDocument      @relation(fields: [invoiceDocumentId], references: [id], onDelete: Restrict)
  link     InvoiceDeliveryLink?

  @@index([status, leasedUntil])
  @@index([invoiceId, createdAt])
  @@map("invoice_deliveries")
}

/// The revocable token behind a LINK delivery. Only a hash is stored, for the
/// same reason the OTP challenge stores only a hash: this row is readable by
/// anything that can read the database, and a live token is a working
/// credential against a patient's bill.
model InvoiceDeliveryLink {
  id         String    @id @default(uuid()) @db.Uuid
  deliveryId String    @unique @map("delivery_id") @db.Uuid
  tokenHash  String    @unique @map("token_hash")
  expiresAt  DateTime  @map("expires_at")
  revokedAt  DateTime? @map("revoked_at")
  openCount  Int       @default(0) @map("open_count")

  delivery InvoiceDelivery @relation(fields: [deliveryId], references: [id], onDelete: Cascade)

  @@index([expiresAt])
```

```
  @@map("invoice_delivery_links")
}
```

Transport contract changes

These are the two narrow, deliberate widenings the epic requires.

```
// channel-gateway.types.ts
export type SendChannelDocumentRequest = {
  externalChatId: string;
  fileName: string;
  mimeType: string;
  content: Uint8Array;
  /** Text shown with the document. Never carries clinical content. */
  caption?: string;
};

// whatsapp-gateway.service.ts — the port gains its second member.
// Satisfiable by GOWA (POST /send/file, multipart), by WAHA (sendFile), and by
// the official Cloud API (document message) — which is the test any new member
// of this port has to pass.
abstract sendDocument(request: SendChannelDocumentRequest): Promise<void>;

// common/mail/mail.types.ts
export type MailAttachment = {
  readonly fileName: string;
  readonly mimeType: string;
  readonly content: Uint8Array;
};
// SendMailRequest gains: readonly attachments?: readonly MailAttachment[];
```

The existing cross-adapter conformance suite (`whatsapp-gateway-contract.spec.ts`) is extended to drive `sendDocument` through both GOWA and WAHA from one fixture table, so D-CS-01's “swapping the gateway is configuration” claim stays checkable rather than asserted.

> **PIN THE WIRE FORMAT IN THE SPIKE**
>
> GOWA documents `POST /send/file` ; the exact multipart field names must be read off its `docs/openapi.yaml` and asserted in the adapter spec rather than assumed from `/send/image` .

API surface & RBAC

METHOD	PATH	PERMISSION	NOTES
`POST`	`/api/v1/invoices/:id/deliveries`	`invoice.deliver:any`	Channels + optional shape override
`GET`	`/api/v1/invoices/:id/deliveries`	`invoice.read:any`	The timeline
`POST`	`/api/v1/invoice-deliveries/:id/retry`	`invoice.deliver:any`	Requeues a FAILED row
`POST`	`/api/v1/invoice-deliveries/:id/revoke`	`invoice.deliver:any`	Kills the link token
`GET`	`/api/v1/patients/:patientId/delivery-consents`	`patient.read:any`	Per-channel state
`PUT`	`/api/v1/patients/:patientId/delivery-consents`	`patient.update:any`	Capture or withdraw at the counter
`GET`	`/inv/:token`	**public route**	Not under `/api/v1`. Rate-limited, no enumeration, serves a presigned GET or an expiry page

> **THE ONLY UNAUTHENTICATED SURFACE IN THIS PRD**
>
> `GET /inv/:token` is `@PublicRoute()`, rate-limited per IP and per token, returns an identical response for expired, revoked and unknown tokens, and never reveals whether an invoice exists.

PERMISSION KEY	ADMIN	DOCTOR	PATIENT
`invoice.deliver:any`	✔	—	—

One new key, deliberately separated from `invoice.write:any`: issuing a bill and transmitting a patient's charges outside the building are different acts, and a clinic that wants a junior cashier to do the first but not the second should be able to express that without a code change.

Edge cases & failure modes

CASE	BEHAVIOUR
Patient shares a WhatsApp number with a family member	The verified link is per (channel, chat, phone) and carries a `patientId`; delivery targets the link for *this* patient, and the caption names the patient so a shared device shows who the bill is for.
Invoice voided after a link was emailed	`FR-E4-15` revokes the link; the page says the invoice is no longer valid.
GOWA session lost (QR re-scan needed)	Deliveries queue rather than fail; the admin channel-health screen already surfaces session state.
WhatsApp bans the clinic number	`WA_GATEWAY_KIND` switches to WAHA, or to a Cloud API adapter when it exists — the port is what makes this configuration (D-CS-01).
Bulk resend after an outage	The daily cap and the shared pacing queue bound the burst; interactive replies keep priority.
Two worker replicas pick the same delivery	Lease claim under `SELECT … FOR UPDATE SKIP LOCKED` — the SATUSEHAT outbox fix applied here from the start.
Consent but no email and no verified number	Both channels disabled with distinct reasons; *Download PDF* remains.

EPIC 5 OF 5 · 47 POINTS

Documents module & approval workflow

> One place where every document the clinic drafts, approves and issues lives — agreements, consent forms, policies, letters, templates and bills — searchable and filterable by type and date, where a drafter names their approvers on the document itself and both parties work it to a decision.

What the module is

The product owner's framing: *"the document is so variant — could be an ordinary document like a patient bill, or a doctor document, or an agreement between patient and doctor, patient and clinic."* So the unit of the feature is not "a template" — it is **a managed document**, of many types, with a lifecycle.

The module is a registry plus a workspace:

- **Registry** — every managed document, with type, title, parties, drafter, status, created/updated/issued timestamps, filter by type, status, drafter and date range, and full-text search over titles and metadata.
- **Workspace** — a document opens to its content, its approval state, its named approvers, its deadline, and the thread of decisions on it. Drafter and approver both act here.

Document types are master data

Types are **a table the clinic manages, not an enum in the schema**. A clinic that issues *Surat Keterangan Sehat* and *Perjanjian Tindakan* should be able to add them without a release, name them in their own words, and set whether each one needs approval.

System types and clinic types

The catch is that **some types are behaviour, not just a label**. Issuing an invoice template publishes a `DocumentTemplateVersion`. Issuing a corpus document releases a file into the chatbot's retrieval set. Code binds to those. A clinic inventing a type cannot invent a handler for it.

So a type row carries a `behavior` discriminator, and the split follows the convention this repo already uses for roles — `isSystem = true`, refused for mutation, because "their shape is owned by `seed.sql`":

	SYSTEM TYPES	CLINIC TYPES
Created by	`seed.sql`	The clinic, at any time
`behavior`	A specific handler — `INVOICE_TEMPLATE`, `CLINIC_CORPUS`, `PATIENT_BILL`	Always `GENERIC`
Clinic may rename	✔	✔
Clinic may set approval policy	✔	✔
Clinic may change `code` or `behavior`	✖	✖ (fixed at `GENERIC`)
Clinic may delete	✖ — deactivate only	✔ when unused, deactivate otherwise

A clinic can never create a type that publishes a template or feeds the chatbot. `behavior` is not a field on the create form; it is set to `GENERIC` by the service and is not accepted from a request. That is the whole safety boundary of making types dynamic, and it is the same shape as the storage service deriving `ownerType` rather than accepting it.

The seeded system types are the nine in the previous revision. Everything beyond them is the clinic's to define. Note that this is **not** the seeded-role question from `OQ-1`: no *role* is seeded here either, but a type whose behaviour has code behind it must exist before that code can run.

What a type carries

Each field answers a question a clinic actually asks when defining a document type, which is why they are columns rather than a settings blob:

FIELD	QUESTION IT ANSWERS
`code`, `name`, `description`	What is this called, in our words? `code` is the stable machine identity; `name` is display and is freely editable
`isApprovalRequired`	Does this need sign-off before we issue it?
`allowSelfApproval`, `requiredApprovals`	How strict is that sign-off?
`requiresPatient`, `requiresDoctor`	Must a document of this type name a patient? A doctor? An agreement does; a policy does not
`contentMode`	Is this drafted in the editor, uploaded as a file, or either? Agreements ship as `EITHER` — RQ-4 confirmed a clinic wants both a template they fill in and a scan of a signed copy
`defaultApproverIds`	Who usually approves this? Pre-fills the drafter's picker without taking the choice away
`isActive`, `sortOrder`	Is it still in use, and where does it sit in the picker?

The approval policy therefore **lives on the type row** — the separate

a type that is itself a row has no reason to be a second row.

The seeded system types

TYPE	WHAT IT IS	APPROVABLE	APPROVAL DEFAULT
`AGREEMENT_PATIENT_CLINIC`	Terms between the patient and the clinic — financial responsibility, general treatment consent	✔	**On**
`AGREEMENT_PATIENT_DOCTOR`	Terms between the patient and a named doctor — procedure-specific agreements	✔	**On**
`CONSENT_FORM`	Informed consent, procedure-specific	✔	**On**
`CLINIC_POLICY_SOP`	Internal policy and standard operating procedures	✔	**On**
`LETTER`	*Surat* the clinic issues — referral cover letters, certificates, correspondence	✔	On
`INVOICE_TEMPLATE`	The E1 invoice layout	✔	**Off**
`CLINIC_CORPUS_DOCUMENT`	A document the chatbot retrieves and cites (`/admin/clinic-corpus`)	✔	**On**
`PATIENT_BILL`	A rendered invoice PDF (E1 output)	✖	n/a — generated at issue, never drafted
`OTHER`	Anything else the clinic manages	✔	Off

we issued this month"* without pretending a generated artefact goes through drafting. It is listed, searchable and openable; it has no draft state and no approver.

What the module does not absorb

Three document surfaces stay where they are, and the reasons are the same ones that shaped E2 and E3:

STAYS PUT	WHY
Patient clinical documents (E2)	They belong to a patient's record and are read in the encounter, next to the vitals and the diagnosis. Pulling them into a clinic-wide registry would make a lab result a browsable list item rather than part of one person's chart
Doctor vault documents (E3)	Private to their owner. A clinic-wide document list that included them would be exactly the admin-browsable view E3 exists to prevent. A doctor sees their own vault documents in their own view and nowhere else
The retrieval corpus itself	The chunks and embeddings stay in the document store. The module manages the *document* and its approval; ingestion remains the corpus's own pipeline

Every row the registry shows is still filtered by whoever is entitled to see that document. The module is one surface over several sources, never a bypass of any source's access rule.

Approval: the drafter names the approvers

The earlier draft of this epic pre-assigned approvers per document kind in a settings screen. The product owner's answer replaced that:

> **NOTE**
>
> *"we can just add approver/reviewer inside the document when drafting the document, and we put everyone except for patients."*

That is the better model and it is simpler to build. The clinic still decides **whether** a type needs approval; the drafter decides **who** approves this particular document, at the moment they know what it is.

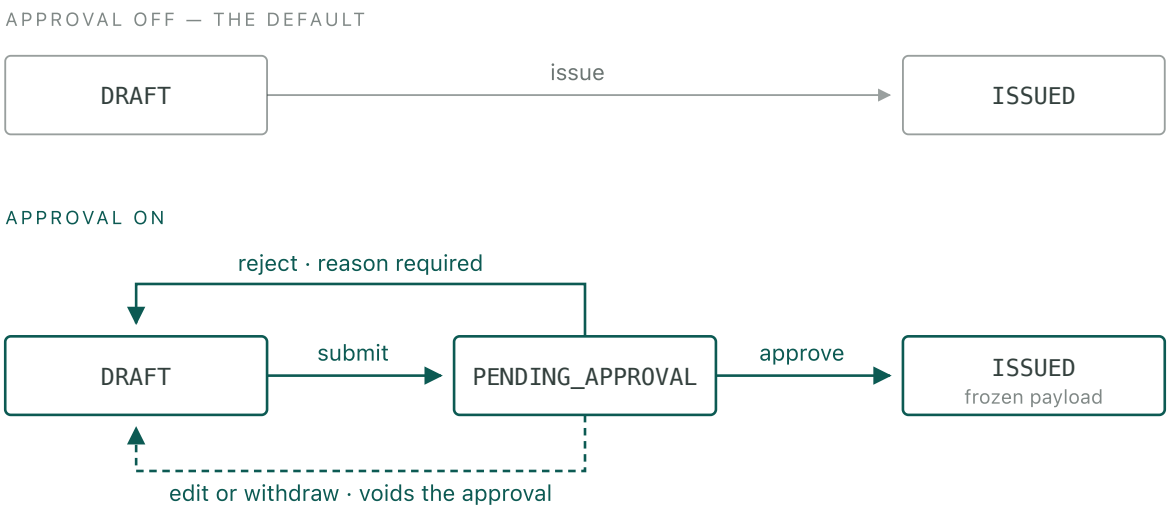
DECISION	WHO MAKES IT	WHERE
Does this *type* need approval at all?	Clinic admin, once	On the type row in Documents → Types (§7.5.2)
Who approves *this* document?	The drafter, per document	On the document, while drafting
When is it due?	The drafter, per document	On the document, while drafting
Approve or reject?	The named approvers	The document workspace

Any staff user can be named as an approver — **every role except** `PATIENT`. There is no separate approver registry to maintain and no per-division mapping to keep in step with the org chart; the org structure already exists for other purposes and does not need to become an approval routing table.

Propose as **D-028** in `docs/post-mvp/decisions.md`.

The lifecycle

The same state machine serves every approvable type; only what gets frozen and what "issue" does differ — for an invoice template it publishes a version, for a corpus document it releases the file to the ingestion worker.



With approval on, there is no edge from `DRAFT` straight to `ISSUED` — that missing arrow is the whole feature. Approving issues the payload frozen at submission, in the same transaction — a template version, or a corpus document released to the ingestion worker.

Functional requirements

Document types (master data)

ID	PRI	REQUIREMENT
FR–E5–31	MUST	Document types are **rows a clinic manages**, not a fixed enum. An admin can create, rename, describe, reorder, activate and deactivate types from a Documents → Types screen.
FR–E5–32	MUST	A clinic-created type always has `behavior = GENERIC`. **`behavior` is never accepted from a request** — the service sets it, so no clinic-defined type can publish a template version or release a file into the retrieval corpus.
FR–E5–33	MUST	Seeded types carry `isSystem = true`. Their `code` and `behavior` are immutable and they cannot be deleted, following the rule `rbac.service.ts` already applies to seeded roles. Their **name, description and approval policy remain editable** — a clinic may call the invoice template whatever it likes.
FR–E5–34	MUST	Approval settings live on the type row: `isApprovalRequired`, `allowSelfApproval`, `requiredApprovals`. There is no separate policy table.
FR–E5–35	MUST	A type declares whether a document of that type must name a **patient** and/or a **doctor**, and whether its content is **drafted, uploaded, or either**. The document form is built from those flags.
FR–E5–36	MUST	A type in use cannot be deleted — the API refuses it and offers deactivation. Deactivating removes it from the new-document picker; existing documents keep their type and stay readable.
FR–E5–37	MUST	Renaming a type changes display everywhere and breaks nothing: `code` is the stable identity that reports, filters and any future integration key on.
FR–E5–38	SHOULD	A type may carry **default approvers** that pre-fill the drafter's picker. The drafter can always change them — it is a convenience, not the routing table `OQ–14` rejected.
FR–E5–39	SHOULD	Type usage counts on the settings screen, so an admin can see which types are actually used before pruning the list.

The registry

ID	PRI	REQUIREMENT
FR–E5–01	MUST	A **Documents** module lists every managed document the viewer is entitled to see, showing type, title, status, drafter, approvers, and created / updated / issued timestamps.
FR–E5–02	MUST	Filters: type, status, drafter, approver, and date range on created or issued.
FR–E5–03	MUST	Search over title, document number and party names. Search never returns a document the viewer could not otherwise open, and result counts do not leak the existence of ones they cannot.
FR–E5–04	MUST	Every row respects its own source's access rule. A doctor's vault document appears only for its owner; a patient bill appears only to someone holding `invoice.read`. **The module is a surface, never a bypass.**
FR–E5–05	MUST	A document detail view shows content, parties, full timestamp history, approval state, named approvers, deadline, and every decision with its reason.
FR–E5–06	SHOULD	Saved filters, so "agreements awaiting my approval" is one click from the sidebar.
FR–E5–07	SHOULD	Export the filtered list as CSV for a survey or audit — metadata only, never document contents in bulk.

Drafting and approval

ID	PRI	REQUIREMENT
FR-E5-08	MUST	An admin sets, **per `ManagedDocumentType`**, whether approval is required. Defaults follow §7.5.2 — agreements, consents, policies and corpus documents on; invoice templates off.
FR-E5-09	MUST	While drafting, the drafter names **one or more approvers** from any staff user. Patients can never be named.
FR-E5-10	MUST	The drafter sets an optional **approval deadline** on the document.
FR-E5-11	MUST	With approval required, a document can only reach `ISSUED` through `PENDING_APPROVAL`. The service refuses a direct issue, not merely the UI.
FR-E5-12	MUST	With approval not required for the type, the drafter issues directly, with no approval UI on the document at all.
FR-E5-13	MUST	Approving requires `document-approval.decide:any`, **separate from** the write permission on the document. Being named as an approver and holding the permission are both required.
FR-E5-14	MUST	**A drafter cannot approve their own document**, even if they named themselves. A clinic with one eligible person may enable `allowSelfApproval`; it defaults to off, enabling it is audited, and a persistent warning shows while it is on.
FR-E5-15	MUST	Submitting freezes the document content and the approver set. Editing either while pending returns it to `DRAFT`, supersedes the request, and notifies the approvers.
FR-E5-16	MUST	Approval issues the document, releasing the frozen version, in one transaction. For an invoice template that publishes a version; for a corpus document it sets `ingestStatus = PENDING`.
FR-E5-17	MUST	Rejection requires a reason. The document returns to `DRAFT`, the reason reaches the drafter, and it stays in the document's history.
FR-E5-18	MUST	The drafter can withdraw a pending submission.
FR-E5-19	MUST	**A corpus document is not ingested until approved.** Under an active policy, confirming an upload leaves `ingestStatus = NOT_APPLICABLE`; approval releases it. An unapproved document is never in the retrieval candidate set.
FR-E5-20	MUST	**Changing `visibility` on an issued corpus document requires re-approval** — it is the field that decides whether a document can be quoted to a patient.
FR-E5-21	MUST	Before deciding, an approver sees the **frozen submission** — for a template, the fixture-data preview; for any uploaded document, the file itself.
FR-E5-22	SHOULD	A diff against the currently issued version, so "what changed" is not reconstructed by hand.
FR-E5-23	SHOULD	Bulk approve on the filtered list for low-risk types, so onboarding a 40-document corpus is not 40 round trips.
FR-E5-24	COULD	`requiredApprovals > 1` for a document that wants two signatures.

Notification and deadlines

ID	PRI	REQUIREMENT
FR-E5-25	MUST	On submission, every named approver is notified **in the in-app bell feed and by email**, both carrying the document, the drafter and the deadline.
FR-E5-26	MUST	The drafter is notified in-app and by email on approval, rejection or supersede.
FR-E5-27	MUST	Where a deadline is set, approvers are reminded as it approaches and again when it passes. An overdue document is flagged in the registry and counted in the sidebar badge.
FR-E5-28	MUST	A missed deadline **never auto-approves and never auto-rejects**. It escalates attention; it does not decide. An approval nobody made must never exist.
FR-E5-29	MUST	Every transition is audited: type policy change, submission, approver assignment, withdrawal, approval, rejection, and the enabling of self-approval.
FR-E5-30	SHOULD	Email uses the same SMTP transport and clinic profile as E4, so approval mail carries the clinic's identity.

User stories

US-E5-01 Drafter writes an agreement and routes it

8 pts

> As a **clinic admin**, I want to draft a patient–clinic agreement and send it to the people who should check it, so that it is reviewed before we ask a patient to sign it.

- **Given** the Documents module **When** the drafter creates an `AGREEMENT_PATIENT_CLINIC`, names the medical director and the finance lead as approvers, sets a deadline of Friday, and submits **Then** the document moves to `PENDING_APPROVAL`, its content and approver set are frozen, and both approvers get an in-app notification and an email naming the document, the drafter and the deadline.
- **Given** a pending document **When** the drafter edits its content **Then** they are warned that editing withdraws the submission; on confirming, it returns to `DRAFT` and both approvers are notified it was superseded.
- **Given** a type whose approval setting is off **Then** the drafter sees an *Issue* action and no approver field anywhere.

US-E5-02 Approver finds their work and decides

8 pts

> As an **approver**, I want everything waiting on me in one filtered list, so that I am not hunting through screens.

- **Given** three documents naming this approver **When** they open Documents and apply *Awaiting my approval* **Then** all three are listed with type, title, drafter, deadline and age, and the sidebar badge shows 3.

- **Given** a document past its deadline **Then** it is flagged overdue in the list, and it is **still pending** — the deadline changed nothing about its state.
- **Given** the approver clicks *Approve* **Then** the frozen version is issued in the same transaction, the drafter is notified in-app and by email, and the decision is recorded with the approver and timestamp.
- **Given** the drafter named themselves and `allowSelfApproval` is off **Then** the service refuses their approval regardless of what the UI offered.

`US-E5-03` Approver rejects with a reason

`3 pts`

> As an **approver**, I want to send a document back with an explanation, so that the drafter knows what to fix.

- **Given** a pending agreement **When** the approver rejects it with "Clause 4 contradicts our refund policy" **Then** the document returns to `DRAFT`, the drafter is notified in-app and by email carrying that reason, and the reason stays in the document's history.
- **Given** a rejection with an empty reason **Then** it is refused — the reason is mandatory.

`US-E5-04` Anyone finds a document later

`5 pts`

> As a **records officer**, I want to find every agreement we issued in a date range, so that I can answer a surveyor without opening a filing cabinet.

- **Given** a registry with 400 documents of six types **When** the officer filters to agreements issued in Q3 and searches a patient's name **Then** matching documents are listed with type, title, parties, drafter, approvers and issue date.
- **Given** a document the officer is not entitled to see **Then** it is absent from both results and counts.
- **Given** the filtered list **When** they export CSV **Then** they get metadata only — never the documents themselves.

`US-E5-05` Corpus documents are gated before the chatbot sees them

`5 pts`

> As a **clinic admin**, I want a new FAQ document checked before the assistant answers from it.

- **Given** approval on for `CLINIC_CORPUS_DOCUMENT` **When** an admin uploads an SOP with visibility `BOTH` and submits **Then** `ingestStatus` stays `NOT_APPLICABLE`, the document shows as pending, and **the chatbot cannot retrieve it**.

- **Given** approval **Then** `ingestStatus` becomes `PENDING`, the worker picks it up, and only then does it enter the corpus.
- **Given** an issued corpus document whose visibility changes from `DOCTOR` to `BOTH` **Then** it returns to `PENDING_APPROVAL` and leaves the retrieval candidate set until re-approved.

`US-E5-06` Nothing changes for a type that needs no approval

`2 pts`

> As a **clinic owner**, I want invoice templates to keep publishing in one click, so that a governance feature does not slow down the thing that did not need it.

- **Given** `INVOICE_TEMPLATE` approval off (the default) **Then** publishing works exactly as E1 specifies, with no approver field, banner or badge.
- **Given** the entitlement `governance.document-approval` disabled **Then** the approval settings and every approval control are absent, and the registry still lists and searches documents.

Data model delta

```
/// What a type's issue step actually does. A bounded enum even though types
/// themselves are unbounded: a clinic can invent a document type, it cannot
/// invent a handler. Clinic-created types are always GENERIC.
enum DocumentTypeBehavior {
  /// Draft or upload -> approve -> issue. Nothing else happens.
  GENERIC
  /// Issue publishes a DocumentTemplateVersion (E1).
  INVOICE_TEMPLATE
  /// Issue sets ingestStatus = PENDING, releasing the file to the worker.
  CLINIC_CORPUS
  /// Generated by E1 when an invoice is issued; never drafted, never approved.
  PATIENT_BILL
}

enum DocumentContentMode {
  DRAFTED
  UPLOADED
  EITHER
}

/// Master data. Seeded rows carry `isSystem = true` and are refused for
/// structural mutation exactly as seeded roles are (`rbac.service.ts`): their
/// `code` and `behavior` are owned by seed.sql because code binds to them.
/// Everything a clinic actually wants to change — the name, the approval
/// policy, the ordering — stays editable on every row.
///
/// The approval policy lives here rather than in a second table: a policy
/// about a type that is itself a row has no reason to be another row.
model DocumentType {
  id                String              @id @default(uuid()) @db.Uuid
  /// Stable machine identity. Reports, filters and future integrations key on
  /// this, which is why renaming is free and re-coding is not.
```

```
  code               String              @unique
  name               String
  description        String?
  /// Never accepted from a request — the service sets GENERIC for anything a
  /// clinic creates. This one line is what makes dynamic types safe.
  behavior           DocumentTypeBehavior @default(GENERIC)
  isSystem           Boolean              @default(false) @map("is_system")

  isApprovalRequired Boolean              @default(false) @map("is_approval_required")
  allowSelfApproval  Boolean              @default(false) @map("allow_self_approval")
  requiredApprovals  Int                  @default(1) @map("required_approvals")

  /// Shape of a document of this type: which parties it must name and whether
  /// its body is written here or uploaded.
  requiresPatient    Boolean              @default(false) @map("requires_patient")
  requiresDoctor     Boolean              @default(false) @map("requires_doctor")
  contentMode        DocumentContentMode  @default(EITHER) @map("content_mode")

  isActive           Boolean              @default(true) @map("is_active")
  sortOrder          Int                  @default(0) @map("sort_order")
  createdAt          DateTime             @default(now()) @map("created_at")
  updatedAt          DateTime             @updatedAt @map("updated_at")
  deletedAt          DateTime?            @map("deleted_at")

  documents        ManagedDocument[]
  defaultApprovers DocumentTypeApprover[]

  @@index([isActive, sortOrder])
  @@map("document_types")
}

/// Pre-fills the drafter's approver picker (FR-E5-38). Deliberately a default
/// and not a rule: the drafter may remove anyone here and add anyone else.
model DocumentTypeApprover {
  id         String @id @default(uuid()) @db.Uuid
  typeId     String @map("type_id") @db.Uuid
  approverId String @map("approver_id") @db.Uuid

  type     DocumentType @relation(fields: [typeId],     references: [id], onDelete: Cascade)
  approver User         @relation(fields: [approverId], references: [id], onDelete: Cascade)

  @@unique([typeId, approverId])
  @@map("document_type_approvers")
}

enum ManagedDocumentStatus {
  DRAFT
  PENDING_APPROVAL
  ISSUED
  ARCHIVED
}

/// The registry row. It carries what the module lists, filters and searches on;
/// the payload lives either inline (`contentHtml`, for a drafted document) or in
/// object storage (`storageKey`, for an uploaded one), never both.
///
/// `subjectTemplateId` / `subjectDocumentId` / `subjectInvoiceId` link a
/// registry row to the thing it governs where one already exists — the
/// `Invoice.encounterId` / `admissionId` pattern of nullable FKs with a CHECK,
/// so a managed document can never point at two subjects and a real foreign key
```

```
/// stops it outliving what it describes.
model ManagedDocument {
  id             String                @id @default(uuid()) @db.Uuid
  /// A row in `DocumentType`, not an enum value — the clinic owns this list.
  typeId         String                @map("type_id") @db.Uuid
  status         ManagedDocumentStatus @default(DRAFT)
  title          String
  documentNumber String?               @map("document_number")
  /// Drafted content, sanitised on every write like a template body.
  contentHtml    String?               @map("content_html")
  /// Uploaded payload. Exactly one of contentHtml / storageKey is set.
  storageKey     String?               @map("storage_key")

  /// Parties, for agreements and consents. All nullable — a policy has none.
  patientId String? @map("patient_id") @db.Uuid
  doctorId  String? @map("doctor_id")  @db.Uuid

  subjectTemplateId String? @map("subject_template_id") @db.Uuid
  subjectDocumentId String? @map("subject_document_id") @db.Uuid
  subjectInvoiceId  String? @map("subject_invoice_id")  @db.Uuid

  draftedById String   @map("drafted_by_id") @db.Uuid
  issuedAt    DateTime? @map("issued_at")
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")
  deletedAt   DateTime? @map("deleted_at")

  /// Restrict, so a type that documents point at cannot be deleted out from
  /// under them — the API surfaces this as "deactivate instead" (FR-E5-36).
  type     DocumentType              @relation(fields: [typeId],    references: [id], onDelete: Restrict)
  patient  PatientProfile?           @relation(fields: [patientId], references: [id], onDelete: Restrict)
  doctor   DoctorProfile?            @relation(fields: [doctorId],  references: [id], onDelete: Restrict)
  requests DocumentApprovalRequest[]

  /// The registry's own query shapes: list by type and status, sort by date,
  /// and find a patient's agreements.
  @@index([typeId, status, createdAt])
  @@index([status, issuedAt])
  @@index([patientId])
  @@index([draftedById])
  @@map("managed_documents")
}

enum DocumentApprovalStatus {
  PENDING
  APPROVED
  REJECTED
  WITHDRAWN
  /// The document or its approver set changed while pending. Distinct from
  /// WITHDRAWN: "the drafter changed their mind" and "the artefact under review
  /// no longer exists" are different facts, and only one is a workflow problem.
  SUPERSEDED
}

/// One round of review. `frozenPayload` is the content and the approver set as
/// submitted — an approver approves a specific artefact reviewed by a specific
/// panel, and changing either is a new round.
model DocumentApprovalRequest {
  id            String                @id @default(uuid()) @db.Uuid
  documentId    String                @map("document_id") @db.Uuid
```

```
  frozenPayload Json                   @map("frozen_payload")
  status        DocumentApprovalStatus @default(PENDING)
  submittedById String                 @map("submitted_by_id") @db.Uuid
  submittedAt   DateTime               @default(now()) @map("submitted_at")
  /// Set by the drafter. Nothing happens automatically when it passes — it
  /// drives reminders and the overdue flag, never a decision (FR-E5-28).
  dueAt         DateTime?              @map("due_at")
  resolvedAt    DateTime?              @map("resolved_at")

  document  ManagedDocument            @relation(fields: [documentId], references: [id], onDelete: Cascade)
  approvers DocumentApprovalApprover[]
  decisions DocumentApprovalDecision[]

  /// Partial unique index in the migration allows at most one PENDING request
  /// per document, so a double submit cannot open two rounds.
  @@index([status, dueAt])
  @@index([documentId, submittedAt])
  @@map("document_approval_requests")
}

/// Who the drafter named on this round. A table rather than an array so the
/// "awaiting my approval" query is an indexed join instead of a JSON scan.
model DocumentApprovalApprover {
  id         String @id @default(uuid()) @db.Uuid
  requestId  String @map("request_id") @db.Uuid
  approverId String @map("approver_id") @db.Uuid

  request  DocumentApprovalRequest @relation(fields: [requestId],  references: [id], onDelete: Cascade)
  approver User                    @relation(fields: [approverId], references: [id], onDelete: Restrict)

  @@unique([requestId, approverId])
  @@index([approverId])
  @@map("document_approval_approvers")
}

model DocumentApprovalDecision {
  id         String   @id @default(uuid()) @db.Uuid
  requestId  String   @map("request_id") @db.Uuid
  approverId String   @map("approver_id") @db.Uuid
  isApproved Boolean  @map("is_approved")
  /// Required when `isApproved` is false, enforced by a CHECK in the migration.
  reason     String?
  decidedAt  DateTime @default(now()) @map("decided_at")

  request  DocumentApprovalRequest @relation(fields: [requestId],  references: [id], onDelete: Cascade)
  approver User                    @relation(fields: [approverId], references: [id], onDelete: Restrict)

  @@unique([requestId, approverId])
  @@map("document_approval_decisions")
}

// NotificationType gains:
//   DOCUMENT_APPROVAL_REQUESTED   DOCUMENT_APPROVAL_APPROVED
//   DOCUMENT_APPROVAL_REJECTED    DOCUMENT_APPROVAL_SUPERSEDED
//   DOCUMENT_APPROVAL_DUE_SOON    DOCUMENT_APPROVAL_OVERDUE
```

API surface

METHOD	PATH	PERMISSION
GET	`/api/v1/documents`	`managed-document.read:any`
POST	`/api/v1/documents`	`managed-document.write:any`
GET	`/api/v1/documents/:id`	`managed-document.read:any`
PATCH	`/api/v1/documents/:id`	`managed-document.write:any`
POST	`/api/v1/documents/:id/submit`	`managed-document.write:any`
POST	`/api/v1/documents/:id/withdraw`	`managed-document.write:any`
POST	`/api/v1/documents/:id/issue`	`managed-document.write:any`
GET	`/api/v1/documents/:id/history`	`managed-document.read:any`
GET	`/api/v1/documents/export`	`managed-document.read:any`
GET	`/api/v1/document-approvals?assignedToMe=true`	`document-approval.decide:any`
POST	`/api/v1/document-approvals/:id/approve`	`document-approval.decide:any`
POST	`/api/v1/document-approvals/:id/reject`	`document-approval.decide:any`
GET	`/api/v1/document-approvals/pending-count`	`document-approval.decide:any`
GET	`/api/v1/document-types`	`managed-document.read:any`
POST	`/api/v1/document-types`	`document-type.write:any`
PATCH	`/api/v1/document-types/:id`	`document-type.write:any`
DELETE	`/api/v1/document-types/:id`	`document-type.write:any`
PUT	`/api/v1/document-types/:id/default-approvers`	`document-type.write:any`

the party and content flags, `isActive` and `sortOrder` — and **never `behavior` or, on a system row, `code`**. `DELETE` refuses a type that has documents and returns a message naming deactivation instead.

row, so the same call returns different sets to different callers — that filtering is the module's core invariant (`FR-E5-04`), not a view concern.

Two existing routes gain behaviour rather than being removed:

document.

RBAC

KEY	NOTES
`managed-document.read:any`	See and search the registry — still filtered per row by the source's own rule
`managed-document.write:any`	Draft, edit, submit, withdraw and issue
`document-approval.decide:any`	Approve or reject. **Separate from write** — that separation is the control
`document-type.write:any`	Manage document types: create, rename, set approval policy, activate, reorder. Replaces the policy-write key — the policy is now a field on the type

Per the product owner's answer to `OQ-1`, **no new role is seeded**. The permission keys ship in `seed.sql` so they exist to be assigned; the clinic composes roles — a cashier, a medical director, a reviewer — through the portal's existing role management, which already audits `ROLE_CREATED` and `ROLE_PERMISSIONS_CHANGED`.

Being **named on a document** and **holding** `decide` are both required to approve. Naming grants nothing on its own, so a drafter cannot manufacture an approver out of someone the clinic never trusted with the permission.

Edge cases & failure modes

CASE	BEHAVIOUR
Deadline passes with no decision	Reminders fire, the document is flagged overdue, the sidebar counts it — and it stays `PENDING_APPROVAL`. Nothing auto-decides (`FR–E5–28`)
A named approver is deactivated or loses `decide`	They drop out of the panel; if none remain, the document is flagged "no eligible approver" and the drafter is prompted to name another
Two approvers decide simultaneously	Row lock on the request; first wins, second is told it was already decided
The drafter names only themselves, self-approval off	Submission is refused at submit time with a clear reason, not discovered at decision time
A document is deleted while pending	Soft delete cascades the request to `SUPERSEDED`; history is retained
Approval turned off for a type while requests are pending	Those return to `DRAFT` with a notification — never silently auto-issued
A corpus document was ingested before the policy was turned on	It stays ingested. Enabling a policy gates future issues and does not pull the corpus out from under a running chatbot — the product owner's answer to `OQ–18`
A patient bill in the registry	Listed, searchable, openable by anyone holding `invoice.read`; no draft state, no approver, no submit action
A clinic deletes a type that has documents	Refused with `409` naming the count, and the response offers deactivation. Documents never lose their type
A clinic deactivates a type mid-draft	Drafts of that type stay editable and submittable; the type simply disappears from the new-document picker
A clinic renames "Invoice template" to something else	Allowed and harmless — `code` is what the invoice-template handler resolves on, never the name
Someone tries to create a type with `behavior = CLINIC_CORPUS`	The field is not in the create schema; the service sets `GENERIC`. A request carrying it is rejected by the Zod DTO, not silently ignored
Two types given the same name	Allowed — names are the clinic's. `code` is unique and is generated from the name with a collision suffix

Non-functional requirements

ID	REQUIREMENT
`NFR-PERF-01`	Invoice PDF render p95 < 2.5 s, p99 < 5 s for a 20-line, ≤ 3-page invoice. Render is async-capable: the UI polls and never blocks payment recording.
`NFR-PERF-02`	Repeat download of an already-rendered document is served from the stored object — no re-render, p95 < 500 ms to mint the URL.
`NFR-PERF-03`	Patient document list for a patient with 200 documents returns p95 < 800 ms, paginated at 25.
`NFR-PERF-04`	Invoice deliveries share the WhatsApp send queue with conversation replies and **never take priority over them**. An interactive reply dispatches ahead of any queued invoice send; invoice sends additionally obey a configurable daily cap.
`NFR-PERF-05`	A requested delivery reaches SENT within 5 minutes at p95, assuming the gateway is connected. Queue depth and oldest-queued age are exported metrics.
`NFR-SEC-01`	Template HTML is sanitised **server-side** on every write against an element and attribute allowlist. `script`, `iframe`, `object`, `embed`, `link`, event handlers, `javascript:` URLs and remote `url()` in inline styles are stripped. Client-side sanitisation is a convenience, never the control.
`NFR-SEC-02`	Widening the bucket MIME allowlist to images ships **in the same change** as the `sharp` re-encode step. The two must not be separate PRs.
`NFR-SEC-03`	The PDF renderer runs in its own container with outbound network access denied and no access to the database or the app's credentials. Template HTML reaches it fully self-contained.
`NFR-SEC-04`	Every download URL carries attachment disposition and a pinned, validated content type. No stored file renders inline in the app or API origin.
`NFR-SEC-05`	Upload-URL and download-URL minting are rate-limited per user — minting is cheap for us but grants storage writes.
`NFR-SEC-06`	Delivery link tokens are ≥ 128 bits of CSPRNG entropy, stored only as a hash — the `ChannelOtpChallenge` rule: a live token is a working credential against a patient's bill. Single-invoice scoped, expiring, revocable. `GET /inv/:token` is rate-limited per IP and per token and returns an identical response for expired, revoked and unknown tokens.
`NFR-SEC-07`	The PDF bytes are fetched server-side from the private bucket and streamed to GOWA or SMTP. A bucket URL — presigned or not — is never placed in a WhatsApp message or an email body.
`NFR-SEC-08`	GOWA credentials, the webhook secret and SMTP credentials are secrets, never logged, and the bridge stays on the private network with no published port — the posture the compose file already enforces.
`NFR-SEC-09`	Approval is enforced in the service layer, never only in the UI. A publish or ingest call under an active policy is refused server-side regardless of what the client offered, and self-approval is blocked by the same check that reads the policy.
`NFR-SEC-10`	An unapproved clinic corpus document is excluded from retrieval **by the repository query**, not by ranking — the same rule personal knowledge bases already follow. A document awaiting approval is not in the candidate set at all, so no chatbot answer can cite it.
`NFR-PRIV-01`	Patient documents and rendered invoices are personal data under UU PDP 27/2022. Access is least-privilege, every read is audited, and nothing is bulk-exported without an explicit admin action that is itself audited. A doctor's vault (E3) is stricter still: no permission grants access to it, and the only non-owner read path is a share its owner created for one named person, revocable and logged.
`NFR-PRIV-02`	Only `patient.nikMasked` is exposed to the variable registry. Plaintext national identifiers never enter a rendered document.

ID	REQUIREMENT
NFR-PRIV-03	No document leaves the system without (a) a recorded per-channel delivery consent naming the privacy-notice version in force, and (b) for WhatsApp, a VERIFIED `ChannelPatientLink` for that patient. Both are enforced in the service, not the UI.
NFR-PRIV-04	Outbound message copy carries billing identifiers and amounts only — never a diagnosis, procedure or medication name. A lock-screen preview must not be a clinical disclosure.
NFR-PRIV-05	Delivery destinations are stored masked on the delivery log. The log is a record that a send happened, not a second copy of the patient's contact details.
NFR-PRIV-06	WhatsApp delivery must be named in the patient privacy notice and covered by the processor mapping in `docs/security/ai-vendor-dpa.md` before production enablement. GOWA is self-hosted, so no new processor is added — but the transport crosses Meta's infrastructure, and that is a disclosure, not an implementation detail.
NFR-RET-01	Patient clinical documents inherit the 25-year RME retention floor. Soft delete only; no purge endpoint; a document under legal hold is never a deletion candidate.
NFR-RET-02	Invoice documents are financial records, retained independently of invoice status and never removed by a clinical retention job.
NFR-AUD-01	READ, CREATE, UPDATE and DELETE are audited for every surface here. New business events get their own actions: `INVOICE_DOCUMENT_RENDERED`, `PATIENT_DOCUMENT_RELEASED`, `CREDENTIAL_VERIFIED`, `CREDENTIAL_REJECTED`, `TEMPLATE_PUBLISHED`.
NFR-AUD-02	Delivery events get their own actions: `INVOICE_DELIVERY_REQUESTED`, `INVOICE_DELIVERY_SENT`, `INVOICE_DELIVERY_FAILED`, `INVOICE_DELIVERY_LINK_OPENED`, `INVOICE_DELIVERY_REVOKED`, `DELIVERY_CONSENT_GRANTED`, `DELIVERY_CONSENT_REVOKED`. A patient asking what the clinic sent them must be answerable from the log.
NFR-AUD-03	Approval events get their own audit actions, carrying the `ManagedDocumentType`: `APPROVAL_POLICY_CHANGED`, `APPROVERS_ASSIGNED`, `SELF_APPROVAL_ENABLED`, `APPROVAL_SUBMITTED`, `APPROVAL_WITHDRAWN`, `APPROVAL_SUPERSEDED`, `APPROVAL_GRANTED`, `APPROVAL_REJECTED`. An approved template version links to the decision that released it, so the trail runs from an approver to the receipt a patient received — and for a corpus document, to the answer the chatbot gave.
NFR-I18N-01	Indonesian-first copy with English fallback. Currency as `Rp 1.234.567`; dates as `30 Agustus 2026`; *terbilang* uses standard Indonesian number words including the `se-` forms (seribu, seratus).
NFR-A11Y-01	The rich-text editor is keyboard-operable end to end, including the variable palette. Toolbar controls carry accessible names.
NFR-OBS-01	Metrics: render duration histogram, render failure rate by reason, upload rejection rate by reason, expiry-notification dispatch count. Alert when render failure exceeds 2% over 15 minutes.
NFR-OBS-02	Delivery metrics: send success rate by channel, time-to-SENT histogram, queue depth, retry count, link open rate, opt-out count. Alert on delivery failure rate > 10% over 15 minutes, and on any WhatsApp session disconnect.
NFR-OBS-03	Approval metrics, broken down by `ManagedDocumentType`: pending queue depth, age of the oldest pending request, decision turnaround, rejection rate, and a count of clinics running with `allowSelfApproval` on. Alert when a request has been pending longer than a configurable threshold.
NFR-COMPAT-01	Generated PDFs are PDF 1.7, open correctly in Chrome, Acrobat, macOS Preview and common Android viewers, and embed all fonts.

Dependencies & risks

The left stripe encodes impact: `clay = high`, `amber = medium`, `grey = low`.

ID	RISK	L	I	MITIGATION
R–1	New renderer container adds deployment surface and cost	High	Med	Sidecar is stateless and horizontally trivial; spike it in sprint 1 (`P16–T01`) before committing the epic
R–2	Rich-text HTML becomes an injection vector into the renderer	Med	High	Server-side allowlist sanitisation + network-denied renderer + a security review before E1 ships
R–3	Widening the MIME allowlist to images regresses the SJ-21 posture	Med	High	`sharp` re-encode lands in the same PR; per-surface allowlist, never bucket-wide only
R–4	Clinics build layouts that break on real data — long names, many lines	High	Med	Fixture preview with deliberately hostile sample data; publish-time validation
R–5	Storage growth from scanned documents outpaces current bucket sizing	Med	Med	20 MiB cap, per-clinic usage metric, quota review at 90 days
R–6	*Terbilang* gets Indonesian number words subtly wrong on a legal document	Med	Med	Property-based tests over 0 … 10^{12} plus a fixed table of known-tricky values (1.000, 100.000, 1.100)
R–7	A share is granted for a survey and left open for years	High	Med	Optional expiry on every share (`FR–E3–20`), an owner reminder on old open-ended shares, and a sharing panel that makes standing shares visible rather than buried
R–8	Retroactive template binding confuses users re-downloading a pre-Phase-16 invoice	Med	Low	Explicit render warning and a UI note; never presented as an original document
R–9	The reserved materai area is misread as satisfying the stamp-duty obligation	Low	High	The area is labelled as a placement for a physical or electronic stamp; OQ-3 stays open until legal answers
R–10	**The clinic's WhatsApp number is banned.** GOWA drives a WhatsApp Web session, not the official Business API; document sends to many recipients look more like automation than a booking reply does	Med	High	Daily send cap, pacing, no bulk sends, opt-out honoured; `WA_GATEWAY_KIND` switches to WAHA, and D-CS-01 already names the official Cloud API as the endgame — the port is what keeps that a config change
R–11	Misdelivery: an invoice reaches the wrong person	Med	High	WhatsApp gated on a possession-proven number; email defaults to a revocable link, not an attachment; destinations masked in logs; revoke action in the UI
R–12	Invoice sends starve booking confirmations on the shared WhatsApp queue	Med	Med	Interactive replies dispatch ahead of invoice sends; queue-depth metric and alert
R–13	Adding `sendDocument` widens a port deliberately kept to one method, and a later Cloud API adapter cannot satisfy it	Low	High	The member is accepted only because all three implementations support document messages; the cross-adapter conformance suite covers it from day one
R–14	Patients treat the WhatsApp channel as customer support for billing disputes	High	Low	The message names the counter and the clinic's hours; the customer-service intent orchestrator already hands unrecognised intents to a human
R–15	Approval becomes a bottleneck — the sole approver is on leave and a broken layout cannot be fixed	Med	Med	Withdraw is always available to the author; the built-in system template keeps invoicing working; delegation is `FR–E5–19` and OQ-16
R–16	`allowSelfApproval` is switched on “temporarily” and never switched off	High	Med	Enabling is audited, the settings screen carries a persistent warning while it is on, and the NFR-OBS-03 metric makes it visible

ID	RISK	L	I	MITIGATION
R–17	Approvers rubber-stamp because they cannot see what changed	High	Med	The frozen-snapshot preview is a MUST (FR–E5–13) and the diff against the published version a SHOULD (FR–E5–16); rejection rate is tracked
R–18	Gating corpus ingestion delays the chatbot's knowledge — a clinic onboarding 40 FAQ documents faces 40 decisions and turns the policy off	High	Med	Bulk decisions (FR–E5–22); the policy is per kind, so a clinic can gate templates without gating the corpus
R–19	Enabling the policy is read as retroactive and someone expects already-ingested documents to leave the corpus	Med	Med	Enabling gates future publishes only, stated in the settings copy; a separate explicit admin action sends existing documents for review

08

Rollout plan

1. **Entitlement.** All five epics ship behind `FeatureEntitlement` flags — `billing.invoice-documents`, `clinical.patient-documents`, `clinical.doctor-credentials`, `billing.invoice-delivery`, `governance.document-approval` — following the existing control-plane pattern. A disabled feature overrides every role grant. `billing.invoice-delivery` additionally refuses to enable while `billing.invoice-documents` is off: there is nothing to deliver.
2. **Migration order.** (a) clinic profile + template tables; (b) `Document` column additions and enum values; (c) expiry notices and notification types. Each is additive; none rewrites existing rows.
3. **Backfill.** None. Existing invoices bind a template on first render; existing `DoctorLicense` rows simply have no scan until someone uploads one. Delivery consent starts empty for every patient and is captured at the counter — deliberately **not** inferred from an existing `ChannelPatientLink`, because agreeing to book over WhatsApp is not agreeing to receive bills there.
4. **Sequencing.** E1 and E2 are independent and can run in parallel. E3 depends on E2's image-upload work and starts after it. **E4 depends on E1** — there is no document to deliver until the renderer works. **E5 depends on E1 for the template kind and on the shipped clinic corpus for the document kind**, and is additive on both: because every policy defaults to off, enabling the entitlement changes nothing until a clinic switches a kind on. Enabling a policy gates future publishes only — already-ingested corpus documents are not pulled out from under a running chatbot.
5. **Pilot.** One clinic, one week, with render failure rate and upload rejection rate on a dashboard before wider enablement. E4 gets its own, stricter pilot: **staff-triggered sends only** (auto-send stays off), a low daily cap, and a week of watching WhatsApp session health and the opt-out count before the cap is raised.
6. **Rollback.** Disabling an entitlement hides every surface without a migration. The renderer sidecar can be removed independently; without it, PDF requests fail closed with a clear error and billing continues to work. Disabling `billing.invoice-delivery` stops new

sends and, as a deliberate second step, an admin action revokes outstanding links — killing the flag alone leaves already-sent links live, which is the correct default but must be a conscious choice.

09

Sprint backlog

Two-week sprints, assumed velocity ≈ 24 points. Task IDs continue the existing `P<phase>-T<nn>` scheme.

TASK	STORY	EPIC	PTS	DEPENDS
SPRINT 1 — DE-RISK THE TWO DECISIONS EVERYTHING ELSE RESTS ON · 23 PTS				
P16-T01	**Spike:** PDF renderer sidecar — Gotenberg in compose, HTML→PDF round trip, network-denied, image size measured. Output: a decision record (D-026)	E1	5	—
P16-T02	ClinicProfile model, CRUD API, logo upload, admin settings page	E1	5	—
P16-T03	Upload hardening for images: `sharp` re-encode, per-surface allowlist, 20 MiB cap, tests	E2	8	—
P16-T04	Variable registry: schema, resolver interfaces, *terbilang* with property tests	E1	5	—
SPRINT 2 — THE RENDER PATH END TO END · 21 PTS				
P16-T05	Template models + CRUD + publish/version API + server-side HTML sanitiser	E1	8	T01, T04
P16-T06	Render service: resolve → HTML → sidecar → S3, snapshot on issue, checksum	E1	8	T01, T05
P16-T07	`Document` extension for patient clinical files: columns, enums, constraints, repository	E2	5	T03
SPRINT 3 — FIRST USER-VISIBLE VALUE · 21 PTS				
P16-T08	Patient document API: upload-url, confirm, list, download, patch, delete, audit	E2	8	T07
P16-T09	Rich-text editor in `@hms/ui` (TipTap): toolbar, tables, page break, image	E1	8	—
P16-T10	Invoice PDF download and print from the billing workspace; VOID watermark	E1	5	T06
SPRINT 4 — TEMPLATING BECOMES SELF-SERVICE · 21 PTS				
P16-T11	Variable palette, atomic chips, repeating-block column config in the editor	E1	8	T09, T04
P16-T12	Template preview with hostile fixture data; publish-time validation	E1	5	T06, T11
P16-T13	Patient documents UI: patient tab, upload dialog, list	E2	8	T08
SPRINT 5 — THE CLINICAL PAYOFF · 18 PTS				
P16-T14	Encounter Documents panel + doctor OWN scoping in the ability factory + audit	E2	8	T08
P16-T15	Release-to-patient flag + portal document list	E2	5	T08
P16-T16	`Document` extension for the private vault: `PERSONAL_DOCUMENT` purpose, category enum, filing fields, owner-scoped repository queries	E3	3	T03
SPRINT 6 — CREDENTIALS AND EXPIRY · 21 PTS				
P16-T17	Vault API under `me/personal-documents`: upload-url, confirm, list, download, patch, hard delete, export; guard test asserting no `ANY` key exists for the surface	E3	5	T16
P16-T18	Doctor vault UI: `app/doctor/documents`, upload dialog, category filing, expiry reminders to the owner, not-used-by-the-assistant notice	E3	5	T17
P16-T19	Licence expiry on `DoctorLicense` — admin dashboard, notifications, dedupe table. **Touches no document**	E3	5	—
SPRINT 7 — HARDENING AND PILOT · 13 PTS				
P16-T41	Offboarding: super-admin offboard action distinct from deactivate, `offboardedAt`, hard-coded reduced ability branch, session revocation, login refusal after the window, email notices, scheduled hard delete, admin preview	E3	8	T34
P16-T20	Scheduling warning for expired STR/SIP	E3	3	T19

TASK	STORY	EPIC	PTS	DEPENDS
P16–T34	Sharing engine: `PersonalDocumentShare`, grant/revoke API, `OWN` extended in the ability factory to resolve a live share, `share:own` key, shared-with-me routes, audit and notifications	E3	5	T17
P16–T35	Sharing UI: owner sharing panel with recipients, last-opened and open counts, revoke; *Shared with me* list; expiry and stale-share reminders	E3	3	T34
P16–T21	Security review pass (template sanitiser, renderer isolation, RBAC matrix), pilot enablement, docs	all	8	all
SPRINT 8 — THE TWO TRANSPORTS LEARN TO CARRY A FILE · 21 PTS				
P16–T22	**Spike + build:** WhatsApp document send — `sendDocument` on the gateway port, GOWA `/send/file` and WAHA `sendFile` adapters, wire format pinned against GOWA's `openapi.yaml`, cross-adapter conformance suite extended	E4	8	T06
P16–T23	Mail attachments: `MailAttachment` on `SendMailRequest`, SMTP + log transports, delivery-mail template sharing the clinic profile	E4	5	T02
P16–T24	Delivery consent model, verified-number gate, `BERHENTI` / `STOP` opt-out handling in the inbound normalizer	E4	8	—
SPRINT 9 — DELIVERY BECOMES A PRODUCT · 18 PTS				
P16–T25	`InvoiceDelivery` + `InvoiceDeliveryLink`, tokenised public `GET /inv/:token` route, revoke action, rate limiting	E4	8	T06, T24
P16–T26	Lease-claimed delivery worker with retry/backoff, priority behind interactive replies, daily cap, delivery-status API	E4	5	T22, T23, T25
P16–T27	Billing UI: send dialog with channel picker and consent state, delivery timeline, retry and revoke	E4	5	T25, T26
P16–T37	PDF password protection: encrypt the rendered document at send time, DOB-derived default, configurable source, message copy that names the password without disclosing it	E4	5	T22, T23
P16–T40	Clinical document delivery: release triggers dispatch to the patient and notifies the attending doctor, per-category delivery defaults, clinical message copy	E4	5	T25, T37, T15
P16–T38	Scheduled delivery: `sendAt` on the outbox, cancel and reschedule, re-check consent and invoice state at send time	E4	3	T26
SPRINT 10 — THE DOCUMENTS MODULE · 16 PTS				
P16–T28	Documents registry: `ManagedDocument`, type enum, list/filter/search API with per-row source access rules	E5	8	T05
P16–T29	Approval engine: drafter-named approvers, deadline, state machine, frozen payload, void-on-change, `document-approval.*` keys with guard tests	E5	8	T28
SPRINT 11 — DRAFTER AND APPROVER EXPERIENCE · 13 PTS				
P16–T30	Notifications (in-app **and** email), deadline reminders and overdue flagging, audit actions, approval history	E5	5	T29, T23
P16–T31	Documents module UI: registry with filters and search, document workspace, drafter and approver interaction, sidebar badge	E5	8	T29
SPRINT 12 — INTEGRATIONS AND AGREEMENTS · 18 PTS				

TASK	STORY	EPIC	PTS	DEPENDS
P16–T32	Invoice-template integration: submit/withdraw wiring, publish 409 under policy, frozen-snapshot preview, diff	E5	5	T29, T31
P16–T33	Clinic-corpus integration: gate ingestion, visibility-change re-approval, retrieval excludes unapproved, bulk decisions	E5	8	T29, T31
P16–T36	Agreement and consent document types: parties, drafted-or-uploaded content, patient linkage	E5	5	T28

Totals: E1 = 57, E2 = 42, E3 = 37, E4 = 52, E5 = 52, cross-cutting = 8 — about 248 points across 12–14 sprints.

Two epics moved after the decisions landed. **E4 went 39 → 47**: password protection on every attachment and scheduled delivery are both real work, and both came out of one answer. **E5 went 37 → 47**: it stopped being an approval policy over two template kinds and became a documents module with a registry, search and a document type nobody had specified yet. That is the honest cost of the reframe, and it is the epic most worth cutting scope from if the phase runs long — the registry and the approval engine are the core; agreements, bulk decisions and CSV export are not.

> **EXPLICITLY NOT IN THIS PLAN**
>
> e-Meterai, inline preview, patient self-upload, OCR, DICOM, merged-PDF credential export (`FR-E3-14`), template import/export (`FR-E1-16`), the hard scheduling block (`FR-E3-15`), SMTP bounce handling (`FR-E4-17`), patient email verification (`FR-E4-18`), auto-send on payment (`FR-E4-14` — built, but shipped **off** and not enabled during the pilot), multi-approver thresholds (`FR-E5-23`), approver delegation (`FR-E5-24`) and scheduled publish (`FR-E5-25`). Each is a COULD or a Non-Goal, and each is dropped first if the phase runs long.
>
> **If the phase must be cut, cut E4 last-in / first-out as a unit.** Its six tasks are the only ones that transmit patient data outside the building, and a partial E4 — sends without consent enforcement, or without the delivery log — is worse than no E4.

Definition of Done — every task

Lint, typecheck, unit tests, integration tests against real Postgres, build, `prisma validate` , both Docker images build — the existing CI gate. Plus: new Zod schemas live in `@hms/shared-types` ; `pnpm api:contract:sync` has been run and the regenerated Orval client is committed; new permission keys are in `seed.sql` with a covering guard test; any new upload surface answers every row of `docs/security/file-uploads.md` .

Definition of Ready — before a story enters a sprint

Acceptance criteria written and agreed; data model delta reviewed; permission keys named; UI placement agreed; any dependency on `P16-T01` or `P16-T03` already merged.

10

Decisions & remaining questions

All eighteen questions from the previous revision are answered. Each is recorded here with what it changed, so nobody re-opens a settled point in sprint planning.

Decisions taken

#	QUESTION	DECISION	WHAT IT CHANGED
`OQ-1`	A seeded cashier role?	**No.** Roles and permissions are composed in the portal	Permission *keys* still ship in `seed.sql` so they can be assigned; **no new role is seeded** in any epic
`OQ-2`	Thermal 80 mm paper?	**No.** Clinics print on lightweight sheet stock	`PaperSize` is A4 / A5 / Letter; `THERMAL_80MM` and its edge cases are gone (`FR-E1-05`)
`OQ-3`	e-Meterai?	**Physical stamp for now**	`FR-E1-13` reserves a placement area only. The broader point — documents are far more varied than invoices — became E5
`OQ-4`	Doctor uploads to a patient record?	**Yes, assigned patients only**	`patient-document.write:own` for DOCTOR resolves to the `DoctorPatient` assignment (`FR-E2-01`)
`OQ-5`	Break-glass access?	**No.** Same rule as `OQ-4`	No emergency path, no justification prompt. Stated explicitly in §7.2.5 so it is not re-proposed
`OQ-6`	Share to roles or named users?	**Named users only**	The role-target COULD is removed from E3
`OQ-7`	Vault offboarding?	**Shares survive; unshared documents stay private and expire; the doctor is warned and offered delete-now**	New §7.3.10. One figure still to confirm — see `RQ-1`
`OQ-8`	Storage sizing?	**Calculated**	New §13.3. Large clinic ≈ 1 TB over 25 years; provision on-prem for 5 years
`OQ-9`	Attachment or link?	**Attachment, password-protected**	Rewrote §7.4.2. Default source is the patient's DOB
`OQ-10`	Same for email?	**Same**	One rule on both channels
`OQ-11`	Daily send cap?	**Unset for now; decide from real analytics**	`FR-E4-16` ships the mechanism off; §7.4.4.1 explains why it exists at all
`OQ-12`	When to move to the Cloud API?	**When the clinic wants a Business account**	A business decision, not a volume threshold (R-10)
`OQ-13`	Auto-send on payment?	**Off.** The front desk asks the patient	`FR-E4-18`
`OQ-14`	How are approvers assigned?	**Named on the document by the drafter**, from any staff user	Replaced the per-kind approver registry. §7.5.4
`OQ-15`	Approval on a first template?	**Answered by** `OQ-14` — it is a per-document choice	No separate rule
`OQ-16`	Approval deadlines?	**Yes, with in-app *and* email notification**	`FR-E5-10` , `FR-E5-25` … `FR-E5-28`
`OQ-17`	Which types are gated?	**Both, but invoices need not be strict**	Per-type defaults in §7.5.2: agreements and corpus on, invoice templates off
`OQ-18`	Retrospective review of the existing corpus?	**No.** Keep what is already ingested	Enabling a policy gates future issues only

Follow-up decisions

#	QUESTION	DECISION	WHAT IT CHANGED
RQ–1	Vault retention after offboarding — 30 months or 30 days?	**30 days**	§7.3.10 rewritten. Because 30 days is short, two things follow: the notice goes **by email** (a departing doctor may never open the portal again), and deactivation opens a **30-day export-only window** so "delete now or wait" is a real choice rather than a countdown
RQ–2	Extend delivery to clinical documents?	**Yes — and send to both ends at once**	New §7.4.5. The patient gets it over WhatsApp or email; the attending doctor is notified and already has it in the encounter panel. Release stays the clinician's gate
RQ–3	Password when a patient has no DOB?	**Front desk captures it at registration — and the null itself gets designed out**	FR–E4–07 refuses a send with no DOB on file. The underlying cause is that a chat booking creates a `PatientProfile` at all; that is addressed in a companion document, First-Timer Registration Flow, which moves chat bookings to a staging record so the demographics columns can become `NOT NULL`
RQ–4	Agreements drafted or uploaded?	**Both**	Agreement types ship with `contentMode = EITHER`; `P16–T36` builds both paths
RQ–5	Is the export-only window a general partial login?	**No — one hard-coded capability set**	Offboarding is a super-admin action distinct from deactivation; the reduced set is a branch in the ability factory keyed on `offboardedAt`, not a role anyone can edit. Sessions are revoked so it applies immediately. §7.3.10.2–3

Remaining questions

#	QUESTION	OWNER	NEEDED BY	BLOCKING?
RQ–7	**Delete-only, or delete plus export, during the window?** Built as the wider reading — a delete-only window would let a doctor destroy their own STR or watch it expire with no way to keep a copy. One line to narrow if that was the intent.	Product owner	Sprint 6	No — FR–E3–23 is the only line affected
RQ–6	**Which patient-document categories dispatch on release by default?** Lab results and radiology are the obvious yes; consent forms and identity documents are probably not.	Clinical lead	Sprint 9	No — FR–E4–27 makes it configurable either way

11

Appendix

Sources consulted

INDONESIAN LICENSING & CREDENTIALING PRACTICE

- Konsil Kedokteran Indonesia — persyaratan STR
- IDI Kotim — perpanjangan Serkom dan STR
- Advomed — panduan lengkap mendapatkan SIP untuk praktik mandiri

- Verisys — healthcare credentialing documents checklist
- Medwave — physician credentialing checklist

WHATSAPP GATEWAY

- GOWA — go-whatsapp-web-multidevice — the bridge already deployed; documents `POST /send/file` alongside `/send/message`
- GOWA API documentation — exact multipart field names must still be pinned from the repo's `docs/openapi.yaml` during `P16–T22`

RECEIPT & STAMP-DUTY PRACTICE

- OnlinePajak — aturan pakai materai 10.000
- Mekari — kwitansi pembayaran: pengertian, jenis, fungsi

INTERNAL

- `AGENTS.md`, `CLAUDE.md`
- `docs/security/file-uploads.md` (SJ-21)
- `docs/ops/rme-retention-policy.md`
- `docs/post-mvp/decisions.md` (D-022 … D-025), `docs/post-mvp/multi-tenancy.md`
- `apps/api/prisma/schema.prisma`, `apps/api/prisma/seed.sql`

Storage sizing (answer to `OQ-8`)

The question was what to provision, and it matters far more for an on-prem deployment than for S3 — on S3 this is a rounding error on the bill, on-prem it is a disk somebody has to buy.

Assumptions. 300 operating days a year. Rendered invoice PDF **150 KB** (text plus an embedded logo, fonts subsetted). Patient clinical document **1.2 MB** — a phone photo after `sharp` re-encode, or a short scanned PDF. Doctor vault **1 MB × 15 documents per doctor**, one-off. Clinic corpus **100 documents × 300 KB**, one-off. **0.3 patient documents per encounter** — not every visit brings paperwork.

TIER	DOCTORS	PATIENTS/DAY	INVOICES/YR	PATIENT DOCS/YR	INVOICES GB/YR	PATIENT DOCS GB/YR	TOTAL
Small	2	30	9,000	2,700	1.3	3.2	**4.5**
Medium	8	100	30,000	9,000	4.3	10.6	**14.8**
Large	20	250	75,000	22,500	10.7	26.4	**37.1**

The headline: a large clinic reaches roughly 1 TB over the 25-year RME retention floor. A small one stays comfortably inside 150 GB. Nothing here is alarming, and that is the useful finding — this feature set does not create a storage problem.

On S3 the cost is negligible. At $0.023/GB/month for S3 Standard: the large clinic pays about **$0.85/month in year one** and about **$21/month** once 25 years have accumulated. Egress is smaller still — a large clinic serving every patient document once is roughly 27 GB/year, a few dollars annually. Lifecycle rules to infrequent-access tiers would cut the mature figure further and are not worth the complexity yet.

On-prem is where the number matters. Provision for usable capacity **× 2** for backup and replica, **× 1.5** if MinIO runs with erasure coding:

TIER	5-YR USABLE	5-YR RAW TO PROVISION	25-YR USABLE	25-YR RAW TO PROVISION
Small	22 GB	**~70 GB**	111 GB	~335 GB
Medium	74 GB	**~225 GB**	371 GB	~1.1 TB
Large	186 GB	**~560 GB**	928 GB	~2.8 TB

Recommendation: size on-prem for five years and plan to grow, rather than buying 25 years of disk on day one. A 1 TB volume covers a medium clinic for its first five years with generous headroom, and disks get cheaper faster than this data accumulates.

What actually drives the number. Patient documents are ~70% of the total, so the two assumptions worth revisiting before anyone buys hardware are the average file size and the documents-per-encounter rate:

AVERAGE PATIENT-DOCUMENT SIZE	LARGE CLINIC, 25-YR USABLE
0.5 MB	543 GB
1.2 MB	**928 GB**
2.0 MB	1.37 TB
3.0 MB	1.92 TB

DOCUMENTS PER ENCOUNTER	LARGE CLINIC, 25-YR USABLE
0.1	488 GB
0.3	**928 GB**
0.6	1.59 TB
1.0	2.47 TB

Two levers follow directly. **Compress on ingest** — the `sharp` re-encode in

whether the average is 0.5 MB or 2 MB; that single setting moves a large clinic's 25-year total by nearly a terabyte. And **if consent forms become per-visit**, the rate goes from 0.3 to near 1.0 and the estimate roughly triples — worth watching once E5's agreements ship.

Migration. A clinic digitising existing paper adds a one-off spike of roughly one year of accumulation per year of history scanned. Five years of back-scanning a medium clinic is

about **50 GB** arriving at once — enough to plan an ingest window for, not enough to change the provisioning tier.

Instrument it rather than trusting it. Ship a per-clinic storage metric with E2 so the real numbers replace these within a quarter of the first deployment. Every figure above is arithmetic over stated assumptions, not measurement.

Glossary

TERM	MEANING
STR	Surat Tanda Registrasi — national registration certificate for a doctor
SIP	Surat Izin Praktik — practice permit, issued per practice location
Serkom	Sertifikat Kompetensi — competence certificate from the kolegium
P2KB / CME	Continuing professional development credits
Kwitansi	Receipt; the document a patient expects after paying
Terbilang	The amount written out in Indonesian words
Materai	Stamp duty affixed to documents above a value threshold
RME	Rekam Medis Elektronik — electronic medical record
UU PDP	UU No. 27/2022, Indonesia's personal data protection law
GOWA	go-whatsapp-web-multidevice — the self-hosted WhatsApp Web bridge this deployment runs
WAHA	The alternative WhatsApp bridge kept as a tested fallback under D-CS-01
JID	A WhatsApp address, e.g. `628123456789@s.whatsapp.net` — the canonical chat id
BERHENTI	“Stop” — the Indonesian opt-out keyword honoured on the WhatsApp channel
Approver	A user holding `document-approval.decide:any` **and** assigned to that kind's approver list — both are required
Share	An owner-created, revocable grant of read access to one vault document for one named person — the relationship that makes `OWN` resolve for a non-owner
Approvable	A clinic-owned thing with a publish step: today an invoice template or a clinic corpus document
Clinic corpus	The clinic-owned document set (`ownerType = CLINIC`) the chatbot retrieves and cites from, managed at `/admin/clinic-corpus`
Frozen payload	The approval-relevant fields captured at submission — content and settings for a template, title/visibility/language for a corpus document; what an approver approves and what publishing then releases

---

Markdown source of record: `docs/product/prd-billing-invoice-and-clinical-documents.md`. Requirement IDs (`FR-E1-09` , `NFR-SEC-02` , `US-E2-02`) are stable — cite them in tickets, tests and PR descriptions so the backlog and the spec point at the same sentence.